

# Prompt-aligned Gradient for Prompt Tuning

Beier Zhu<sup>1</sup> Yulei Niu<sup>2</sup> Yucheng Han<sup>1</sup> Yue Wu<sup>3</sup> Hanwang Zhang<sup>1\*</sup>

<sup>1</sup>Nanyang Technological University <sup>2</sup>Columbia University <sup>3</sup>Damo Academy, Alibaba Group  
beier002@e.ntu.edu.sg, yn.yuleiniu@gmail.com yucheng002@e.ntu.edu.sg  
matthew.wy@alibaba-inc.com hanwangzhang@ntu.edu.sg

## Abstract

Thanks to the large pre-trained vision-language models (VLMs) like CLIP [37], we can craft a zero-shot classifier by discrete prompt design, e.g., the confidence score of an image being “[CLASS]” can be obtained by using the VLM provided similarity between the image and the prompt sentence “a photo of a [CLASS]”. **Furthermore, prompting shows great potential for fast adaptation of VLMs to downstream tasks if we fine-tune the soft prompts with few samples.** However, we find a common failure that improper fine-tuning or learning with extremely few-shot samples may even under-perform the zero-shot prediction. Existing methods still address this problem by using traditional anti-overfitting techniques such as early stopping and data augmentation, which lack a principled solution specific to prompting. **In this paper, we present Prompt-aligned Gradient, dubbed ProGrad to prevent prompt tuning from forgetting the general knowledge learned from VLMs.** In particular, **ProGrad only updates the prompt whose gradient is aligned (or non-conflicting) to the general knowledge, which is represented as the optimization direction offered by the pre-defined prompt predictions.** Extensive experiments under the **few-shot learning, domain generalization, base-to-new generalization and cross-dataset transfer settings** demonstrate the stronger few-shot generalization ability of ProGrad over state-of-the-art prompt tuning methods.

## 1. Introduction

After seeing and reading countless image-text association pairs, large and deep vision-language models (VLM) [37, 18] can memorize the **general knowledge** (a.k.a. encyclopedic knowledge) about what visual patterns correspond to what textual sequence and vice versa. Thanks to the powerful language modeling of VLMs, we can establish a communication channel in human-readable natural language, i.e., **prompt** [25, 52, 19], to query the general knowledge.

Prompting bridges the interface gap between the pre-trained and downstream tasks (e.g., regression vs. classification) without the need for additional fine-tuning adaptation. For example, we can craft a concrete prompt—“a photo of a [CLASS]”—for zero-shot image classification: by using the popular vision-language model CLIP [37], we input the image to the vision end and the prompt sentence to the language end, then obtain a vision-language similarity as the confidence score of classifying the image as “[CLASS]”.

In practice, the prompt-based zero-shot image classification is not accurate because the hand-crafted prompt may not be the most machine-favorable (e.g., “this is a picture of” could be more grammatically prevailing in VLM training), or not specific to the downstream domain (e.g., “a photo of a person doing” is better in action recognition) [37]. **Recently, prompt tuning or prefix tuning [23, 26, 55, 56] has been proposed to replace the hand-crafted prompt with a set of tunable word embedding vectors, which do not have to be translatable back to human-readable words.** Yet, prompt tuning is still as tricky as conventional fine-tuning: as the training continues, the generalization ability may decrease and even under-perform the zero-shot baseline. As shown in Figure 1(a&b), the prompt tuning method CoOp [55] achieves the best results via early stopping, and its accuracies heavily drop by at most 4% when the training continues. Besides, Figure 1(c&d) show that CoOp underperforms zero-shot CLIP without augmentation or enough samples from downstream tasks. To the best of our knowledge, existing methods still rely on the conventional anti-overfitting techniques such as early stopping and data augmentation [55, 56, 11, 36], which lacks a principled solution to the nature of improper prompt tuning. Furthermore, the Grad-CAM visualization results indicate that the fine-tuned prompt **misleads the VLM to forget the general knowledge** that the classification should at least focus on the foreground object but not the background. Comparing CoOp (Figure 2(b)) with zero-shot CLIP (Figure 2(c)), we find that the CoOp model **distracts its attention to the background**, while CLIP mainly focuses on the foreground object. These results demonstrate the over-fitting risk of existing prompt

\*Corresponding author.

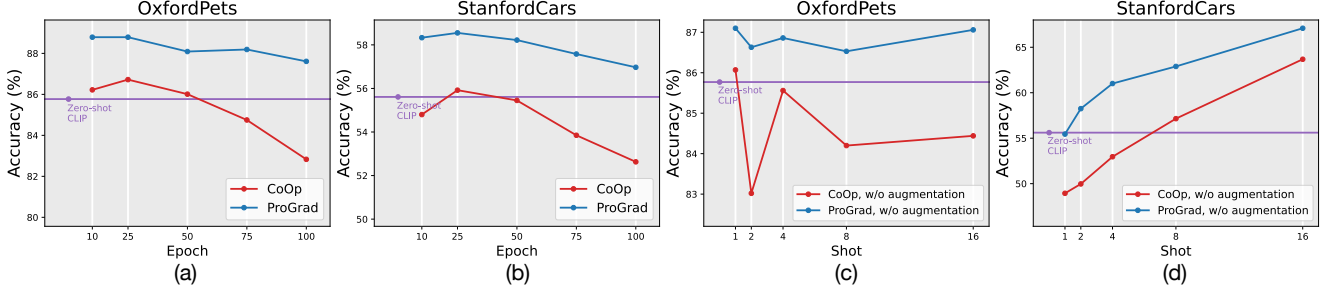


Figure 1: Comparison of Zero-shot CLIP, CoOp, and our ProGrad on Stanford Cars and OxfordPets datasets. (a)&(b): Given 1 shot training sample, CoOp’s performance severely drops and under-performs zero-shot CLIP by large margins when the training continues. (c)&(d): CoOp may fail to improve CLIP without data augmentation or plenty of samples.

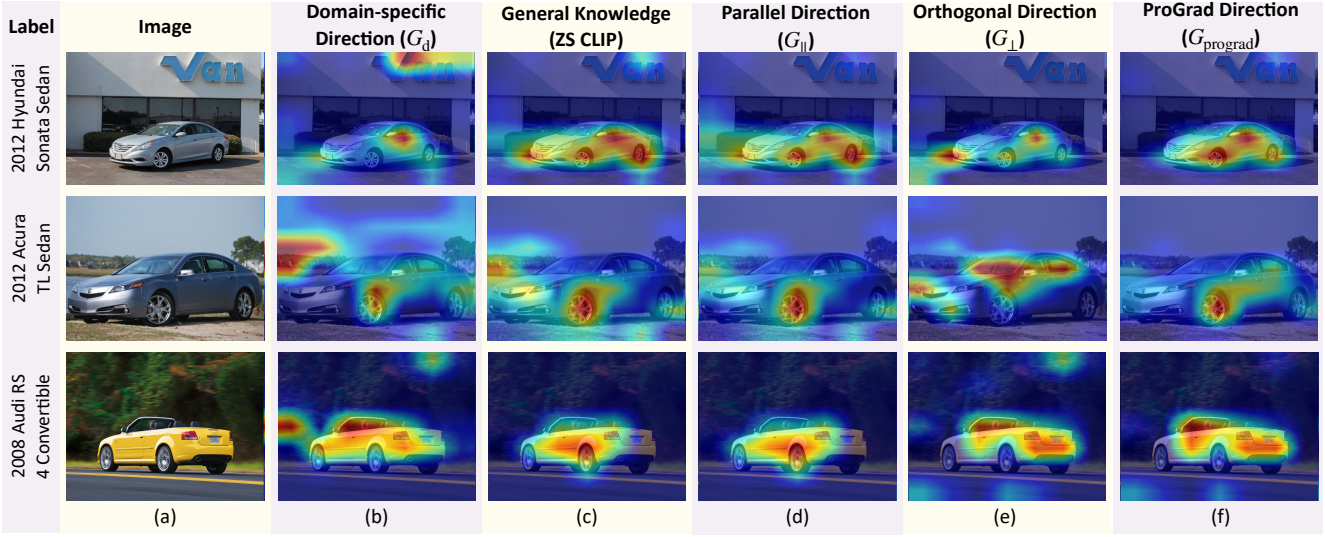


Figure 2: Comparisons of Grad-CAM [43] visualization for prompt tuning methods using different gradient strategies on Stanford Cars Datasets.

tuning strategies, especially when the number of training samples is extremely limited (e.g., 1 or 2).

To this end, we present a novel prompt tuning method called Prompt-aligned Gradient (ProGrad) to overcome the improperly biased tuning for CLIP. The principle of ProGrad is to regularize each tuning step not to conflict with the general knowledge offered by the original prompt, e.g., the zero-shot CLIP predictions. Specifically, we measure the general knowledge direction  $G_g$  using the gradient of Kullback–Leibler (KL) divergence between the predictions of the zero-shot prompted CLIP and the few-shot fine-tuned model, which we name as *general direction*. Similarly, we compute the domain-specific knowledge direction  $G_d$  using the gradient of cross-entropy between the ground-truth and the few-shot fine-tuned model, dubbed *domain-specific direction*. We decompose the domain-specific direction  $G_d$  into: 1) a vector  $G_{\perp}$  orthogonal to the general direction, which denotes the non-conflicting domain-specific knowl-

edge; and 2) a vector  $G_{\parallel}$  parallel to the general direction, which denotes the general knowledge. Note that the first gradient component does NOT override the general direction as any two orthogonal vectors can be transformed into two non-conflicting base vectors. For the second component, it must be one of the two directions: 1) the same of the general direction, which indicates that the update is aligned to the general knowledge, and 2) the opposite of general direction, indicating a conflicting update that should be discarded to avoid forgetting. Overall, in each iteration, ProGrad only updates the parameters in the prompt-aligned direction that has an acute angle to the general direction. Compared to CoOp and CLIP, both  $G_g$  and  $G_{\perp}$  (Figure 2(d&e)) help to regularize the model to focus on the foreground, and ProGrad (Figure 2(f)) further improves the visual response.

Following CLIP, CoOp and CoCoOp [56], we evaluate ProGrad under the few-shot learning, domain generalization, base-to-new generalization and cross-dataset transfer

settings over 15 image classification benchmarks, covering generic **object classification, fine-grained image recognition, action classification**. In summary, our ProGrad achieves: 1) clear improvement compared to CoOp over all of the 11 datasets; 2) clear improvement on the **harmonic mean of base and new classes accuracies** on all 11 datasets compared to CoOp and CoCoOp, and 3) clear improvement on both the **source and target datasets** of the domain generalization.

## 2. Related Work

**VLMs Adaptation.** VLM can be adapted for various downstream tasks, *e.g.*, visual question answering [21, 48], visual grounding [52], image retrieval [28] and semantic segmentation [39]. We focus on **image classification task**. Conventional “**pre-train then fine-tune**” paradigm that plugs in a classifier to the visual backbone and trained on downstream data has been widely-adopted, *e.g.*, **Linear Probe** [37]. **CLIP-Adapter** [10] add feature adapters to boost fine-tuning results. Recently, NLP community has introduced a “**prompt-based learning**” paradigm that fine-tunes the prompt using a “fill-in-the-blank” cloze test to maximize the ground-truth token [23, 26]. The prompt-based learning has recently been applied in CV community [55, 29, 7, 46, 4, 20, 57]: CoOp [55] uses a continuous prompt optimization from downstream data instead of hand-craft design. CoCoOp [56] further extends CoOp by learning image conditional prompt to improve generalization. ProDA [29] learns a prompt distribution over the output embedding space. VPT [7] introduces variational prompt tuning by combining a base learned prompt with a residual vector sampled from a instance-specific underlying distribution. TPT [46] proposes a test-time prompt tuning, which does not require training data and optimizes the prompt to achieve consistent predictions across different augmented views. Our ProGrad follows the line of *prompt-based learning* to improve few-shot generalization ability by aligning the gradient to general direction, without model structure modification or tuning the pre-trained parameters.

**Knowledge Transfer.** Forgetting mitigation is widely used in incremental learning [27, 40, 35, 42, 17] through knowledge distillation or memory replay. **However, prompt-based learning differs fundamentally from incremental learning in that it does not have access to pre-trained data**, which is required for incremental learning to store old data from memory storage. For example, OGD [8] projects gradients from new classes to the orthogonal direction of previous task gradients, but the requirement to store old task gradients is not possible for prompt tuning since we lack access to the pre-training process. Moreover, OGD modifies gradients of downstream tasks in non-conflicting scenarios, potentially leading to sub-optimal performance. Another related field that leverages gradient matching to transfer knowledge is domain generalization [45, 38] and multi-task learning [44, 53].

However, these methods are not directly applicable in prompt tuning whose **transfer direction is only from general to downstream**. In Appendix, we will show how their methods fail in several ablative studies.

## 3. Methodology

In this section, we introduce the preliminary concepts of prompt-based zero-shot inference, prompt-based learning, and present our proposed Prompt-aligned Gradient solution to align the domain knowledge with general knowledge for few-shot generalization.

### 3.1. Preliminaries

**Contrastive language-image pre-training (CLIP)** [37] adopts a contrastive language-image pre-training paradigm on tremendous pairs of images with natural language descriptions. For contrastive learning, the associated image and sentences are taken as the positive samples, while the non-associated pairs are regarded as negative samples. The contrastive objective maximizes the similarity of positive pairs while minimize the similarity of negative pairs.

**Zero-shot transfer inference** adapts the pre-trained CLIP model to downstream tasks without fine-tuning the model. Taking image classification as an example, zero-shot transfer is enabled by formulating the classification task as an image-text matching problem, where the text is obtained by extending the “[CLASS]” name using a template like “a photo of a [CLASS].”. CLIP [37] finds that such a simple template narrows the distribution gap to pre-training text inputs. The image-class matching score is measured based on the cosine similarity  $\langle \mathbf{w}_i, \mathbf{f} \rangle$  between the image feature  $\mathbf{f}$  and the class-extended text feature  $\mathbf{w}_i$  for  $i$ -th class. The image feature  $\mathbf{f}$  for image  $\mathbf{x}$  is extracted by the image encoder, while the text feature  $\mathbf{w}_i$  for  $i$ -th class is obtained by feeding the prompt description into the text encoder. The probability for  $i$ -th class is obtained as

$$p_{zs}(\mathbf{w}_i|\mathbf{x}) = \frac{\exp(\langle \mathbf{w}_i, \mathbf{f} \rangle / \tau)}{\sum_{j=1}^K \exp(\langle \mathbf{w}_j, \mathbf{f} \rangle / \tau)}, \quad (1)$$

where  $K$  denotes the number of classes, and  $\tau$  is a temperature learned by CLIP.

**Prompt-based learning** further strengths the transferring ability of the CLIP model and avoids prompt engineering by automatically learning the prompt given few samples from the downstream task. Different from the zero-shot transfer that used a fixed hand-craft prompt, CoOp [55] constructs and fine-tunes a set of  $M$  continuous context vectors  $\mathbf{v} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_M\}$  as the turnable prompt. Specifically, the prompt  $\mathbf{t}_i = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_M, \mathbf{c}_i\}$  combines the learnable context vectors  $\mathbf{v}$  and the class token embedding  $\mathbf{c}_i$ , and is fed to the text encoder  $g(\cdot)$ . CoOp optimizes the static context vectors  $\mathbf{v}$  by minimizing the negative log-likelihood

of the ground-truth token:

$$\begin{aligned}\mathcal{L}_{ce}(\mathbf{v}) &= -\sum_i \mathbf{y}_i \log p(\mathbf{t}_i|\mathbf{x}), \\ p(\mathbf{t}_i|\mathbf{x}) &= \frac{\exp(\langle g(\mathbf{t}_i), \mathbf{f} \rangle / \tau)}{\sum_{j=1}^K \exp(\langle g(\mathbf{t}_j), \mathbf{f} \rangle / \tau)},\end{aligned}\quad (2)$$

where  $\mathbf{y}$  denotes the one-hot ground-truth annotation and  $K$  denotes the number of classes.

### 3.2. Prompt-aligned Gradient

CoOp faced a challenge that the transfer performance drops when the number of annotations is very limited (e.g., one per class), even underperforms the zero-shot transfer. Also, CoOp heavily relies on anti-overfitting techniques such as early stopping and data augmentation. To overcome the over-fitting challenge, we propose an effective and efficient fine-tuning paradigm ProGrad to align the few-shot downstream knowledge with the large-scale general knowledge.

Motivated by the success of knowledge distillation [34, 16] in knowledge transfer, we leverage the zero-shot CLIP predictions as the general knowledge, and compare the fine-tuned predictions with the general knowledge to regularize the gradient direction. Specifically, we obtain the domain-specific direction by calculating the cross-entropy  $\mathcal{L}_{ce}(\mathbf{v})$  between the model prediction  $p(\mathbf{t}_i|\mathbf{x})$  and the ground-truth  $\mathbf{y}$  according to Eq. (2), and the general knowledge direction based on the Kullback-Leibler (KL) divergence between  $p(\mathbf{t}_i|\mathbf{x})$  and the zero-shot CLIP prediction  $p_{zs}(\mathbf{w}_i|\mathbf{x})$ :

$$\mathcal{L}_{kl}(\mathbf{v}) = -\sum_i p_{zs}(\mathbf{w}_i|\mathbf{x}) \log \frac{p(\mathbf{t}_i|\mathbf{x})}{p_{zs}(\mathbf{w}_i|\mathbf{x})}. \quad (3)$$

We denote the gradients of  $\mathcal{L}_{kl}(\mathbf{v})$  and  $\mathcal{L}_{ce}(\mathbf{v})$  as  $\mathbf{G}_g = \nabla_{\mathbf{v}} \mathcal{L}_{kl}(\mathbf{v})$  and  $\mathbf{G}_d = \nabla_{\mathbf{v}} \mathcal{L}_{ce}(\mathbf{v})$ , respectively. The relations between  $\mathbf{G}_g$  and  $\mathbf{G}_d$  are two-fold. (1) Their angle is smaller than  $90^\circ$  (Figure 3(a)), which indicates that the optimization direction of few-shot downstream knowledge does not conflict with general knowledge. In this case, we safely set the updated gradient direction  $\mathbf{G}_{prograd}$  as  $\mathbf{G}_d$ . (2) Their angle is larger than  $90^\circ$  (Figure 3(b)), which indicates that the few-shot downstream knowledge conflicts with general knowledge. In other words, optimizing the context vectors following  $\mathbf{G}_d$  will lead to the forgetting of the pre-trained general knowledge. In this case, we project the  $\mathbf{G}_d$  to the orthogonal direction of  $\mathbf{G}_g$  to optimize the model for classification, which avoids increasing the KL loss. Our ProGrad strategy is mathematically formulated as:

$$\mathbf{G}_{prograd} = \begin{cases} \mathbf{G}_d, & \text{if } \mathbf{G}_d \cdot \mathbf{G}_g \geq 0 \\ \mathbf{G}_d - \lambda \cdot \frac{\mathbf{G}_d \cdot \mathbf{G}_g}{\|\mathbf{G}_g\|^2} \mathbf{G}_g, & \text{otherwise.} \end{cases} \quad (4)$$

Fig 3(c) illustrates the pipeline of our ProGrad. Instead of updating the context vectors using  $\mathbf{G}_d$  in CoOp [55], we optimize the context vectors using  $\mathbf{G}_{prograd}$ , which prevent the

gradient direction from overfitting to few-shot downstream samples. We further introduce  $\lambda$  in Eq. (4) to generalize the formulation, which can flexibly control the strength of general knowledge guidance in applications. In particular,  $\lambda = 1$  denotes projecting  $\mathbf{G}_d$  to the orthogonal direction of  $\mathbf{G}_g$  (Figure 3(b)), while setting  $\lambda = 0$  makes ProGrad degenerate to CoOp, i.e., CoOp is a special case of our strategy. We include the detailed analysis of  $\lambda$  in Appendix.

**Generalization Error Analysis.** We further theoretically analyze the generalization error of our ProGrad. Here, we provide a sketch proof and include the detailed justification in Appendix. Our ProGrad keeps the optimal value  $\mathcal{L}_{kl}$  of the pre-trained domain when optimizing the empirical risk on the downstream domain. The model  $\hat{f}_{prograd}$  learned by such update rule can be viewed as optimizing the empirical risk on pre-trained and downstream domains [53]:

$$\hat{f}_{prograd} = \operatorname{argmin}_{f \in \mathcal{F}} \hat{\mathcal{R}}_{(d+p)}(f) = \operatorname{argmin}_{f \in \mathcal{F}} \hat{\mathcal{R}}_d(f) + \hat{\mathcal{R}}_p(f), \quad (5)$$

where  $\mathcal{F}$  is the function class, and  $\mathcal{R}(\cdot)$  and  $\hat{\mathcal{R}}(\cdot)$  denote the expected risk and empirical risk. We bound the generalization error of ProGrad by virtue of Rademacher Complexity [1] and the Theorem 6.2 in [54]. The detailed proof is in Appendix.

**Theorem 1** Let  $\mathbf{X}_1^{N_d} = \{\mathbf{x}_n^{(d)}\}_{n=1}^{N_d}$  and  $\mathbf{X}_1^{N_p} = \{\mathbf{x}_n^{(p)}\}_{n=1}^{N_p}$  be two set of i.i.d. samples drawn from the downstream domain  $\mathcal{D}_d$  and the pre-trained domain  $\mathcal{D}_p$ . Then for any  $\epsilon > 0$ , we have with probability at least  $1 - \epsilon$ ,

$$\begin{aligned}\mathcal{R}_d(\hat{f}_{prograd}) &\leq \hat{\mathcal{R}}_{(d+p)}(\hat{f}_{prograd}) + \frac{1}{2} \gamma_{\mathcal{F}}(D, P) \\ &\quad + \mathfrak{R}_p(\mathcal{F}) + \mathfrak{R}_d(\mathcal{F}) + \frac{3}{2} \sqrt{\frac{\ln(4/\epsilon)}{2N_d}} \\ &\quad + \frac{3}{2} \sqrt{\frac{\ln(4/\epsilon)}{2N_p}} + \frac{1}{2} \sqrt{\frac{\ln(4/\epsilon)}{2}} \left( \frac{1}{N_d} + \frac{1}{N_p} \right),\end{aligned} \quad (6)$$

where  $\gamma_{\mathcal{F}}(D, P)$  is the integral probability metric [31] that measures the difference between the distribution of pre-trained domain and the downstream domain,  $\mathfrak{R}_d(\mathcal{F})$  and  $\mathfrak{R}_p(\mathcal{F})$  are the Rademacher complexity of  $\mathcal{F}$ .

Note that the bound of  $\mathfrak{R}(\mathcal{F})$  is inversely proportional to the number of training samples. Theorem 1 shows that the generalization error  $\mathcal{R}_d(\hat{f}_{prograd})$  is bounded by the empirical training risk  $\hat{\mathcal{R}}_{(d+p)}(\hat{f}_{prograd})$ , the two domain gap  $\gamma_{\mathcal{F}}(D, P)$  and the estimation error. The empirical training risk can be minimized to arbitrary small value when using deep models with high capacity. The estimation error that related to  $N_p$  asymptotically tends to 0 as the sample size  $N_p$  tends to infinity. Thanks to the large amount of pretrained samples

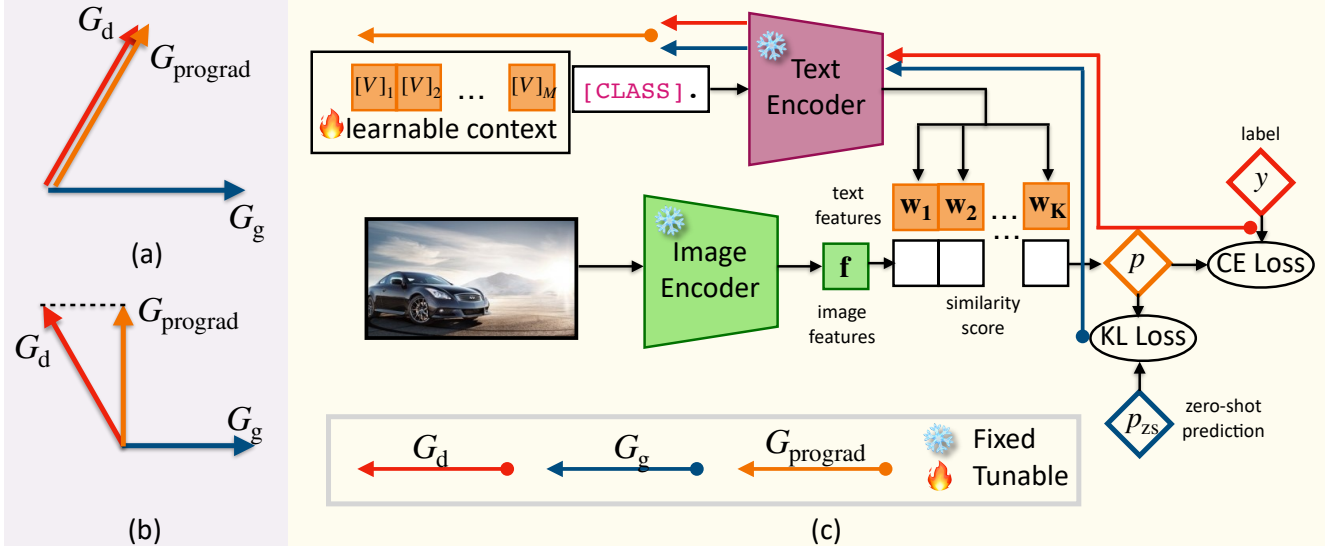


Figure 3: (a) If  $G_d$  is aligned with  $G_g$ , we set  $G_{\text{prograd}}$  as  $G_d$ . (b) If  $G_d$  conflicts with  $G_g$  (i.e., their angle is larger than  $90^\circ$ ), we set  $G_{\text{prograd}}$  as the projection of  $G_d$  on the orthogonal direction of  $G_g$ . (c) Training pipeline of our ProGrad. Only the context vectors are learnable.

$N_p$ , we can approximate the generalization error bound as

$$\begin{aligned} \mathcal{R}_d(\hat{f}_{\text{prograd}}) &\leq \frac{1}{2}\gamma_{\mathcal{F}}(S, P) + \mathfrak{R}_d(\mathcal{F}) \\ &\quad + \frac{3}{2}\sqrt{\frac{\ln(4/\epsilon)}{2N_d}} + \frac{1}{2}\sqrt{\frac{\ln(4/\epsilon)}{2} \frac{1}{N_d}}. \end{aligned} \quad (7)$$

Similarly, we have the generalization error for CoOp  $\hat{f}_{\text{coop}}$  as

$$\mathcal{R}_d(\hat{f}_{\text{coop}}) \leq 2\mathfrak{R}_d(\mathcal{F}) + 3\sqrt{\frac{\ln(4/\epsilon)}{2N_d}} + \sqrt{\frac{\ln(4/\epsilon)}{2} \frac{1}{N_d}}. \quad (8)$$

Under the assumption that the gap between pre-trained and downstream domains  $\gamma(P, D)$  is small, the estimation error bound of  $\mathcal{R}_d(\hat{f}_{\text{coop}})$  is at least two times greater than  $\mathcal{R}_d(\hat{f}_{\text{prograd}})$ . Considering that  $N_d$  is typically very small in few-shot setting, our ProGrad model  $\hat{f}_{\text{prograd}}$  achieves a much lower error bound than conventional fine-tuning model like CoOp  $\hat{f}_{\text{coop}}$ .

## 4. Experiments

### 4.1. Datasets and Implementation Details

We validate the effectiveness of ProGrad on four settings: (1) few-shot classification, (2) domain generalization, (3) base-to-new generalization, (4) cross-dataset transfer.

**Datasets.** For few-shot learning, base-to-new generalization and cross-dataset transfer, we use 11 datasets, i.e., ImageNet [6] and Caltech101 [9] for generic object classification, OxfordPets [33], StanfordCars [22], Flowers102 [32],

Food101 [2] and FGVCAircraft [30] for fine-grained image recognition, EuroSAT [13] for satellite image classification, UCF101 [47] for action classification, DTD [5] for texture classification, and SUN397 [50] for scene recognition. For domain generalization, we use ImageNet as the source dataset and select ImageNet-V2 [41], ImageNet-Sketch [49], ImageNet-A [15], ImageNet-R [14] as the target datasets.

**Training Details.** For few-shot learning, following CoOp and CLIP, all models are trained with  $\{1, 2, 4, 8, 16\}$  shots respectively then evaluated on the full test split. For domain generalization and base-to-new generalization, we evaluate 4-shot performance, which justifies the robustness under low-shots condition. All results of learning-based models are averaged over three random seeds. The standard deviation values can be found in the Appendix. Unless otherwise stated, we adhere to CoOp to use ResNet-50 [12] as the backbone of image encoder. Following [55] and [56], the length of context tokens  $M$  is set to 16 for few-shot classification and  $M = 4$  for the other three settings.  $\lambda$  is set to 1 by default, except that  $\lambda$  is set to 0.8 for 16 shots. We adopt the training settings from CoOp, e.g., training epochs, training schedule and the data augmentation settings, and refer readers to Appendix for further details.

**Baselines.** We compare ProGrad against 4 methods: (1) Zero-shot CLIP (2) Linear probe (3) CoOp and (4) CoCoOp. Although our method can beat some other fine-tune methods, e.g., CLIP-Adapter [10], we focus on single prompt-based learning methods. The results of other fine-tuning methods are in Appendix.

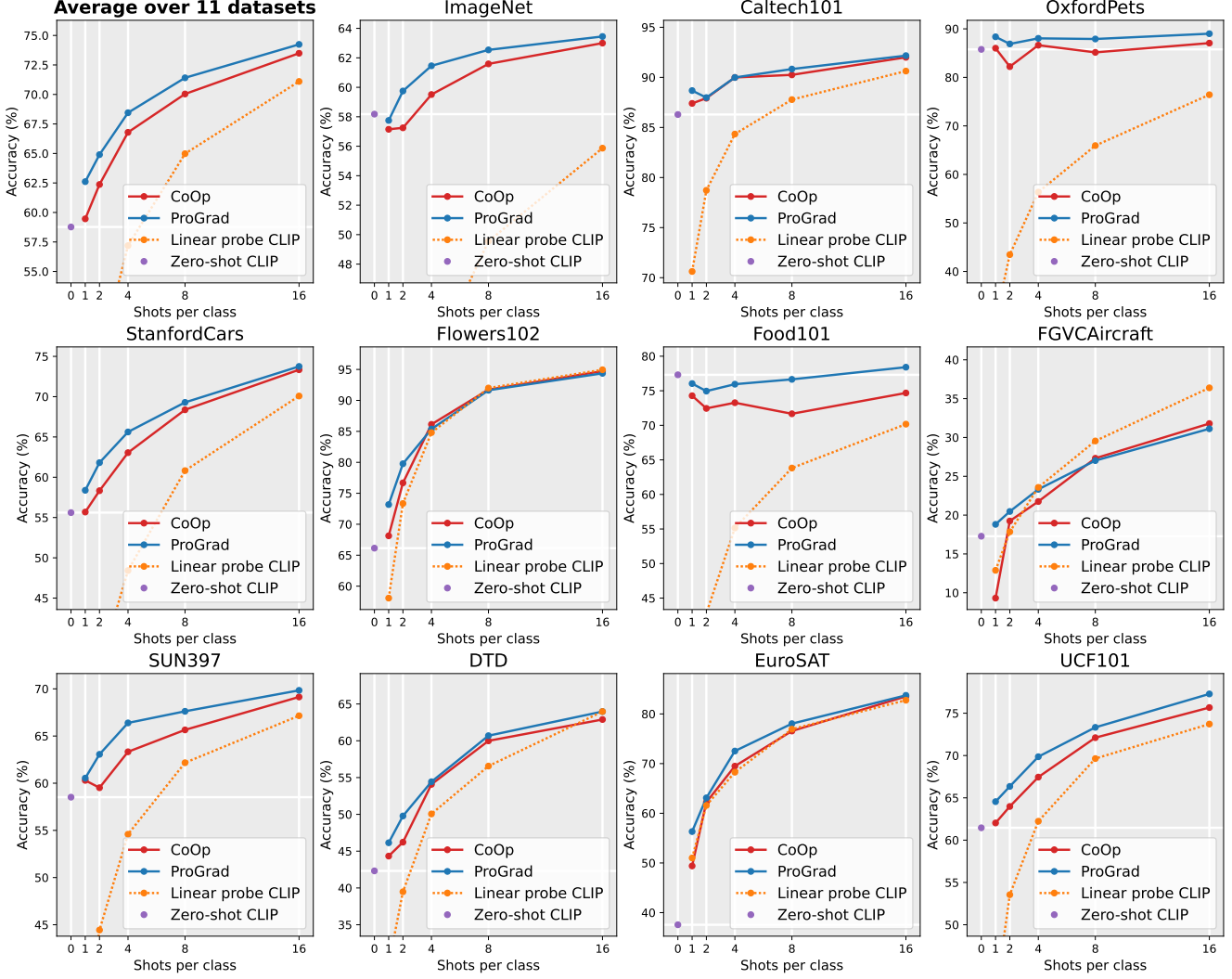


Figure 4: Accuracy (%) of few-shot learning on 11 datasets. The context length  $M$  is set to 16. Standard deviations are reported in Appendix.

## 4.2. Few-Shot Classification

**Setup.** We compare with two zero-shot CLIP models, *i.e.*, CLIP and CLIP++ stand for using single prompt and prompt ensembling respectively (Please refer to Appendix for the prompts templates). ProGrad and ProGrad++ stand for using single prompt and prompt ensembling as general knowledge to implement ProGrad respectively. **Note that we only use the hand-crafted prompt ensembling to generate  $G_g$ , which provides a more accurate general direction.** Therefore, ProGrad++ still optimizes a *single* prompt with 16 learnable tokens, which is identical to the size of CoOp.

Table 1 provides averaged accuracy over 11 datasets, and Figure 4 illustrates the detailed comparisons. Overall, ProGrad clearly outperforms over baselines on average performance. Specifically, ProGrad outperforms CoOp

Table 1: Averaged accuracy (%) of few-shot learning on 11 datasets.  $M = 16$ .

| #shots    | 0            | 1            | 2            | 4            | 8            | 16           |
|-----------|--------------|--------------|--------------|--------------|--------------|--------------|
| CLIP      | 58.77        | -            | -            | -            | -            | -            |
| CLIP++    | <b>59.38</b> | -            | -            | -            | -            | -            |
| LP        | -            | 37.32        | 48.02        | 57.27        | 64.88        | 70.57        |
| CoOp      | -            | 59.46        | 62.37        | 66.79        | 70.04        | 73.49        |
| ProGrad   | -            | 62.61        | 64.90        | 68.45        | 71.41        | 74.28        |
| ProGrad++ | -            | <b>63.06</b> | <b>65.28</b> | <b>68.71</b> | <b>71.80</b> | <b>75.03</b> |

by 9.5%, 6.9% and 5.1% on FGVCaircraft, EuroSAT and Flowers102 given 1 shot, and the average improvement is 3.2%. These results demonstrate the **anti-overfitting ability of our ProGrad when the training samples are extremely**

Table 2: Evaluation on robustness to distribution shift with different visual backbones. Standard deviations are reported in Appendix.

| (a) ResNet50 |              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              | Source       | Target       |              |              |              |
|              | ImageNet     | -V2          | -Sketch      | -A           | -R           |
| CLIP         | 58.18        | 51.34        | 33.32        | 21.65        | 56.00        |
| LP           | 41.29        | 33.65        | 13.09        | 11.18        | 26.82        |
| CoOp         | 61.34        | 53.81        | 32.83        | 22.08        | 54.62        |
| CoCoOp       | 61.04        | 53.71        | 32.30        | 22.07        | 53.60        |
| ProGrad      | <b>62.17</b> | <b>54.70</b> | <b>34.40</b> | <b>23.05</b> | <b>56.77</b> |

| (b) ResNet101 |              |              |              |              |              |
|---------------|--------------|--------------|--------------|--------------|--------------|
|               | Source       | Target       |              |              |              |
|               | ImageNet     | -V2          | -Sketch      | -A           | -R           |
| CLIP          | 61.24        | 54.82        | 38.66        | 28.03        | 64.34        |
| LP            | 47.01        | 38.46        | 19.09        | 16.33        | 39.43        |
| CoOp          | 63.99        | 56.99        | 39.40        | 29.50        | 64.04        |
| CoCoOp        | 63.59        | 56.98        | 39.16        | 29.09        | 64.14        |
| ProGrad       | <b>64.98</b> | <b>57.86</b> | <b>40.53</b> | <b>30.13</b> | <b>65.61</b> |

| (c) ViT-B/32 |              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              | Source       | Target       |              |              |              |
|              | ImageNet     | -V2          | -Sketch      | -A           | -R           |
| CLIP         | 62.00        | 54.75        | 40.82        | 29.59        | 66.01        |
| LP           | 46.77        | 39.12        | 20.32        | 16.32        | 39.48        |
| CoOp         | 64.74        | 56.59        | 40.03        | 31.10        | 64.54        |
| CoCoOp       | 64.63        | 56.59        | 40.74        | 30.27        | 64.12        |
| ProGrad      | <b>65.36</b> | <b>57.42</b> | <b>41.73</b> | <b>31.89</b> | <b>66.53</b> |

| (d) ViT-B/16 |              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              | Source       | Target       |              |              |              |
|              | ImageNet     | -V2          | -Sketch      | -A           | -R           |
| CLIP         | 66.73        | 60.84        | 46.13        | 47.80        | 74.01        |
| LP           | 54.70        | 45.57        | 28.20        | 22.47        | 44.12        |
| CoOp         | 69.86        | 62.83        | 46.90        | 48.98        | 74.55        |
| CoCoOp       | 70.13        | 63.05        | 46.48        | 49.36        | 73.80        |
| ProGrad      | <b>70.45</b> | <b>63.35</b> | <b>48.17</b> | <b>49.45</b> | <b>75.21</b> |

limited. Furthermore, leveraging prompt ensembling can further explore the potential of ProGrad. From Table 1, with more accurate general knowledge offered by prompt ensembling, CLIP++ improves the zero-shot CLIP from 58.77% to 59.38%; ProGrad++ increases the accuracy of ProGrad from 74.28% to 75.03% at 16 shots.

### 4.3. Domain Generalization

We train our ProGrad on the source dataset (ImageNet) for three different random seeds, and assess it on ImageNet-V2, ImageNet-Sketch, ImageNet-A, and ImageNet-R. This

setting evaluates the generalizability on a target domain which differs from the source domain. Fine-tuning on limited data from a specific domain may mislead the model to learn spurious correlations or in-distribution patterns, resulting in biased models with poor performance in unseen domains. In contrast, zero-shot CLIP avoids exploiting such spurious correlations or patterns, as it is not learned on that distribution. By leveraging knowledge from the pre-trained domain to regularize fine-tuning on a specific distribution, our ProGrad method is expected to be robust to distribution shifts. As shown in Table 2, despite the exposure to the source dataset, ProGrad clearly outperforms other methods on all target datasets as well as the source dataset with ResNet-based and ViT-based backbones.

### 4.4. Base-to-New Generalization

We follow CoCoOp [56]

to evaluate the generalization performance from seen classes to unseen classes. All the classes are equally divided into two groups, *i.e.*, base classes and new classes, and all methods are only trained on base classes and tested on both base classes and new classes.

Table 3: Averaged accuracy (%) over 11 datasets for base-to-new generalization.

|         | Base         | New          | H.           |
|---------|--------------|--------------|--------------|
| CLIP    | 61.72        | 65.91        | 63.64        |
| CoOp    | 71.96        | 61.26        | 65.58        |
| CoCoOp  | 72.23        | 60.77        | 65.35        |
| ProGrad | <b>73.29</b> | <b>65.96</b> | <b>69.06</b> |

The harmonic mean of base classes and new classes accuracies is reported to evaluate the trade-off. As illustrated in Table 3, ProGrad yields the highest average performance across all metrics, whereas CoOp and CoCoOp exhibit poor performance for new classes, consistently underperforming zero-shot CLIP. These results highlight that ProGrad’s superior generalizability to both base and new classes. The detailed results of 11 datasets are in Appendix.

### 4.5. Cross-Dataset Transfer

All models are trained on ImageNet as source dataset and evaluated on the rest 10 target datasets. The goal of this setting is to demonstrate the potential to transfer beyond a single dataset. The results are presented in Table 4. As shown, our ProGrad not only achieves the highest performance on source datasets but also outperforms other baselines 9 out of 10 target datasets.

### 4.6. Further Analysis

#### Comparison with Learning without Forgetting (LwF).

ProGrad employs the gradient direction of knowledge distillation loss as a form of regularization. To determine whether our approach is equivalent to conventional knowledge distillation, we compared its performance against a simple knowledge distillation method, *i.e.*,  $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}} + \alpha \cdot \mathcal{L}_{\text{kd}}$ , which is identical to the implementation of Learning without

Table 4: **Comparison of prompt learning methods in the cross-dataset transfer setting.** Prompts are learned from **4-shots** ImageNet.

|         | Source       | Target       |              |              |              |              |              |              |              |              |              |              |
|---------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|         | ImageNet     | Caltech101   | OxfordPets   | StanfordCars | Flowers102   | Food101      | FGVCAircraft | SUN397       | DTD          | EuroSAT      | UCF101       | Average      |
| CoOp    | 61.34        | 84.48        | 85.99        | 54.16        | 60.10        | 75.48        | 14.09        | 57.48        | 35.32        | <b>26.72</b> | 57.56        | 55.70        |
| CoCoOp  | 61.04        | 84.73        | 86.42        | 52.34        | 61.24        | 73.79        | 13.74        | 55.94        | 36.60        | 23.46        | 57.97        | 55.21        |
| ProGrad | <b>62.17</b> | <b>88.30</b> | <b>86.43</b> | <b>55.61</b> | <b>62.69</b> | <b>76.76</b> | <b>15.76</b> | <b>60.16</b> | <b>39.48</b> | 24.87        | <b>58.70</b> | <b>57.36</b> |

Table 5: Comparison with LwF (knowledge distillation). Average accuracy (%) over 11 datasets.

| #shots                   | 1            | 2            | 4            | 8            | 16           |
|--------------------------|--------------|--------------|--------------|--------------|--------------|
| CoOp                     | 59.46        | 62.37        | 66.79        | 70.04        | 73.49        |
| LwF(KD), $\alpha = 0.25$ | 61.09        | 63.01        | 67.74        | 70.90        | 73.39        |
| LwF(KD), $\alpha = 0.5$  | 61.13        | 63.36        | 67.14        | 70.34        | 72.68        |
| LwF(KD), $\alpha = 1$    | 61.52        | 64.07        | 66.52        | 70.01        | 72.01        |
| LwF(KD), $\alpha = 2$    | 60.98        | 62.66        | 64.92        | 67.78        | 68.98        |
| LwF(KD), $\alpha = 4$    | 59.58        | 61.76        | 62.92        | 65.01        | 65.42        |
| ProGrad                  | <b>62.61</b> | <b>64.90</b> | <b>68.45</b> | <b>71.41</b> | <b>74.28</b> |

Table 6: Applying ProGrad to cosine classifier. Average accuracy (%) over 11 datasets.

| #shots    | 1            | 2            | 4            | 8            | 16           |
|-----------|--------------|--------------|--------------|--------------|--------------|
| Cosine    | 30.50        | 43.74        | 53.33        | 61.26        | 65.00        |
| + ProGrad | <b>32.29</b> | <b>46.14</b> | <b>55.18</b> | <b>62.05</b> | <b>66.47</b> |

Forgetting (LwF)[24]. We repeated few-shot experiments on 11 datasets using a range of  $\alpha$  values and report the average results in Table 5. The results demonstrate that ProGrad outperforms KD for various few-shot settings. Although KD (LwF) with small  $\alpha \leq 1$  improves the performance of CoOp in low-shot scenarios (e.g., 1, 2, and 4 shots), its performance drops when the shots is large (i.e., 8 and 16 shots). These findings suggest that ProGrad operates differently from KD (LwF) and is more robust to the sample number.

**Failure cases.** We analyze cases where ProGrad models fail while CoOp succeeds. In specific, we count the percentage of the failure cases that zero-shot CLIP models also fails in Figure 5. We found that a high proportion of the failure cases are also mis-classified by zero-shot CLIP model (red bar in Figure 5), implying that the imprecision of zero-shot general knowledge represented by  $G_g$  is detrimental to model generalization. As sample size increases,  $G_d$  represents downstream knowledge more accurately and with less bias. As expected, the red bar becomes larger.

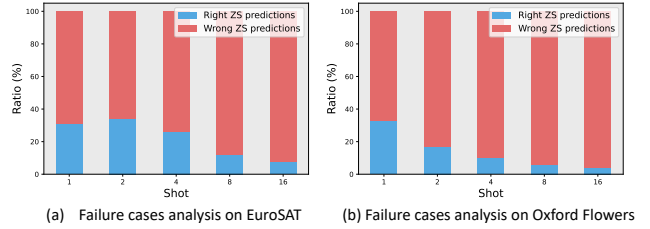


Figure 5: Distribution of samples that are mis-classified by ProGrad but correctly classified by CoOp.

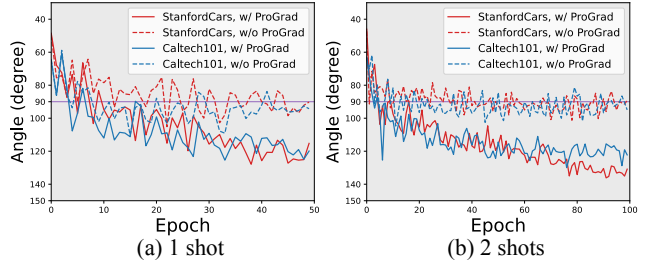


Figure 6: Angles between  $G_d$  and  $G_g$  during training on StanfordCars and Caltech101.

**Conflict of knowledge.** ProGrad requires the updated gradient direction be acute to the general knowledge gradient directions. We explore how this constraint helps to defuse the conflicts of domain-specific and general knowledge by visualizing the angle between their representative gradients during training (angle between  $G_d$  and  $G_g$ ). As depicted in Figure 6, without  $G_{\text{prograd}}$ , the angle between  $G_d$  and  $G_g$  converges to 90 degree due to the fact that “all high-dimensional random vectors are almost always orthogonal to each other” [3]. Intuitively, without any constraint, the optimization direction  $G_d$  is independent to the general direction, and the average angle would be orthogonal. In contrast, utilizing  $G_{\text{prograd}}$  results in the convergence of the angle to an obtuse angle. The reason is that  $G_{\text{prograd}}$  intervenes the model to learn the downstream knowledge aligned with the

general knowledge and leads to the insufficient learning of downstream knowledge that is incompatible with the general knowledge. As training stabilizes,  $G_d$  struggles to learn the conflicting knowledge, reflecting an obtuse angle to the  $G_g$ . Thanks to ProGrad, we discard such conflicting knowledge to avoid forgetting.

### Upper Bound of Performance.

As the general gradient direction  $G_g$  is the key for improvement, we are interested in the upper-bound of performance if we can find an oracle general direction  $G_g^{\text{full}}$  instead of the one offered by hand-crafted prompt. To achieve this, we first optimize a prompt with plain cross-entropy loss on the full dataset to create  $G_g^{\text{full}}$  and then use such gradient to implement ProGrad. The averaged performance over 11 datasets, as shown in Figure 7, indicate a more accurate regularization direction  $G_g^{\text{full}}$  elicits a stronger ProGrad model. The detailed results of 11 datasets are in Appendix.

**Applying ProGrad to conventional fine-tuning.** We are also interested in the effectiveness of ProGrad for the conventional “pre-train then fine-tune” paradigm. Specifically, we plug in an additional cosine classifier on top of the visual backbone and compare the averaged performance over 11 datasets of few-shot classification. Table 6 shows that conventional fine-tuning can benefit from our ProGrad. The implementation details and the result of each dataset are provided in Appendix.

**Effect of hyper-parameter  $\lambda$ .** We further analyze the effect of the hyper-parameter  $\lambda$  described in Eq. (4) in the main paper. Results are shown in Table 7. As discussed in Section 3.2 in the main paper, a smaller  $\lambda$  weakens the general knowledge regularization, which results in a inferior performance under low-shot setting for most datasets. However, for DTD in Table 7, using a smaller  $\lambda = 0.9$  to reduce the general knowledge regularization can improve the 16 shots results. One possible reason is that texture images of DTD has large gap with the CLIP pre-trained images that collected from the Internet, stronger regularization from pre-trained knowledge might be detrimental to the fine-tune performance if downstream data is sufficient.

## 5. Conclusion

In this paper, we pointed out the over-fitting issues of existing prompt tuning methods for few-shot generalization, which heavily relies on early stopping and data augmentation. We proposed a prompt tuning method ProGrad that

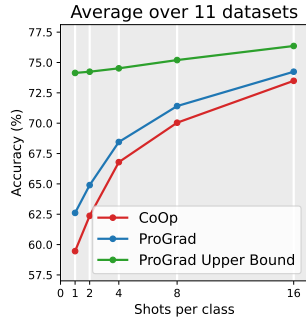


Figure 7: The upper bound of performance (averaged accuracy) of GradPrad.

Table 7: Accuracy (%) of 1, 2, 4, 8, and 16 shots training with different  $\lambda$  on DTD and OxfordPets.

| (a) OxfordPets. |        |         |         |         |          |
|-----------------|--------|---------|---------|---------|----------|
| $\lambda$       | 1 shot | 2 shots | 4 shots | 8 shots | 16 shots |
| 0               | 86.01  | 82.21   | 86.63   | 85.15   | 87.06    |
| 0.2             | 87.12  | 83.16   | 84.87   | 84.00   | 86.15    |
| 0.4             | 88.09  | 83.56   | 85.50   | 84.04   | 86.67    |
| 0.7             | 87.74  | 84.70   | 86.93   | 86.30   | 87.90    |
| 0.9             | 88.26  | 86.47   | 87.52   | 87.38   | 88.52    |
| 1.0             | 88.36  | 86.89   | 88.04   | 87.91   | 89.00    |

| (b) DTD.  |        |         |         |         |          |
|-----------|--------|---------|---------|---------|----------|
| $\lambda$ | 1 shot | 2 shots | 4 shots | 8 shots | 16 shots |
| 0         | 44.33  | 46.22   | 54.08   | 59.99   | 62.89    |
| 0.2       | 43.80  | 45.21   | 54.02   | 60.14   | 63.61    |
| 0.4       | 44.17  | 47.44   | 54.32   | 59.65   | 63.16    |
| 0.7       | 44.93  | 47.77   | 54.92   | 59.28   | 63.10    |
| 0.9       | 45.78  | 48.46   | 55.44   | 60.46   | 64.28    |
| 1.0       | 46.14  | 49.78   | 54.43   | 57.98   | 61.15    |

regularize each tuning step not to conflict with the general knowledge of the hand-crafted prompt. Experiments on few-shot classification, base-to-new generalization, domain generalization and cross-dataset transfer over 11 datasets demonstrate the effectiveness and efficiency of our ProGrad. In the future, we will explore how to apply ProGrad on other tasks like **object detection and segmentation**.

## Acknowledgments

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG2-PhD-2021-01-002) and (AI Singapore AISG2-RP-2021-022). The authors would like to thank the reviewers for their comments that help improve the manuscript.

## A. Justification from Generalization Error

We further analyze the generalization error bound of our ProGrad. We define the expected risk  $\mathcal{R}(\cdot)$  and empirical risk  $\hat{\mathcal{R}}(\cdot)$  of a classifier  $f$  on domain  $\mathcal{D}$  as

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(f(X), Y)], \quad \hat{\mathcal{R}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(X_i), Y_i) \quad (9)$$

where  $\ell(f(X), Y)$  denotes the cross-entropy and  $N$  is the volume of training data. We are interested in the downstream domain  $\mathcal{D}_d$  and pre-trained domain  $\mathcal{D}_p$ , respectively.<sup>1</sup>

<sup>1</sup>The pre-trained dataset includes samples from diverse classes. Here, we only consider the pre-trained data belonging to the classes of downstream task.

Let  $\mathcal{F}$  be a function class, the conventional fine-tune model  $\hat{f}_{\text{coop}}$  is trained on  $\mathcal{D}_d$  by

$$\hat{f}_{\text{coop}} = \underset{f \in \mathcal{F}}{\operatorname{argmin}} \hat{\mathcal{R}}_d(f). \quad (10)$$

The zero-shot CLIP model  $\hat{f}_p$  is considered to be trained on  $\mathcal{D}_p$  by

$$\hat{f}_p = \underset{f \in \mathcal{F}}{\operatorname{argmin}} \hat{\mathcal{R}}_p(f). \quad (11)$$

For the implementation of ProGrad, we initialize the model  $\hat{f}_{\text{prograd}}$  using the pre-trained model  $\hat{f}_p$ . We regularize each training step not to increase the KL divergence between the predictions of  $\hat{f}_{\text{prograd}}$  and  $\hat{f}_p$ . In this way,  $\hat{f}_{\text{prograd}}$  can keep the optimal value of the pre-trained domain  $\mathcal{L}_{\text{kl}}$  when optimizing the empirical risk on the downstream domain. The model  $\hat{f}_{\text{prograd}}$  learned by our ProGrad can be viewed as optimizing the empirical risk on both domains:

$$\hat{f}_{\text{prograd}} = \underset{f \in \mathcal{F}}{\operatorname{argmin}} \hat{\mathcal{R}}_{(d+p)}(f) = \underset{f \in \mathcal{F}}{\operatorname{argmin}} \hat{\mathcal{R}}_d(f) + \hat{\mathcal{R}}_p(f). \quad (12)$$

Based on Theorem 4.1 of [51], assuming that the neural network has  $L$  layers with parameters matrices  $W_1, \dots, W_L$ , and their Frobenius norm are at most  $M_1, \dots, M_L$  and the activation functions are 1-Lipschitz continuous, positive-homogeneous, and applied element-wise. The output of the neural network is the softmax function that predicts  $c$  classes. Let  $\mathcal{F}$  be a function class with the range  $[a, b]$ . Distribution is such that  $\|\mathbf{x}\| \leq B$ . Let  $\mathbf{X}_1^{N_d} = \{\mathbf{x}_n^{(d)}\}_{n=1}^{N_d}$  and  $\mathbf{X}_1^{N_p} = \{\mathbf{x}_n^{(p)}\}_{n=1}^{N_p}$  be two set of i.i.d. samples drawn from the downstream domain  $\mathcal{D}_d$  and the pre-trained domain  $\mathcal{D}_p$ . Then for any  $\epsilon > 0$ , we have with probability at least  $1 - \epsilon$ ,

$$\begin{aligned} \mathcal{R}_d(\hat{f}_{\text{prograd}}) &\leq \hat{\mathcal{R}}_{(d+p)}(\hat{f}_{\text{prograd}}) + \frac{1}{2} \gamma_{\mathcal{F}}(D, P) + \\ &\frac{cB \left( \sqrt{2 \log(2)L} + 1 \right) \prod_{j=1}^L M_j}{\sqrt{N_p}} \\ &+ \frac{cB \left( \sqrt{2 \log(2)L} + 1 \right) \prod_{j=1}^L M_j}{\sqrt{N_d}} \\ &+ \frac{3}{2} \sqrt{\frac{(b-a) \ln(4/\epsilon)}{2N_d}} \\ &+ \frac{3}{2} \sqrt{\frac{(b-a) \ln(4/\epsilon)}{2N_p}} \\ &+ \frac{1}{2} \sqrt{\frac{(b-a)^2 \ln(4/\epsilon)}{2} \left( \frac{1}{N_d} + \frac{1}{N_p} \right)}, \end{aligned} \quad (13)$$

where  $\gamma_{\mathcal{F}}(D, P)$  is the integral probability metric [31] that measures the difference between the distribution of pre-trained domain and the downstream domain. The Eq. (13)

shows that the generalization error  $\mathcal{R}_d(\hat{f}_{\text{prograd}})$  is bounded by the empirical training risk  $\hat{\mathcal{R}}_{(d+p)}(\hat{f}_{\text{prograd}})$ , the two domain gap  $\gamma_{\mathcal{F}}(D, P)$  and the estimation error that is inversely proportional to number of training samples, *i.e.*,  $N_d$  and  $N_p$ . The empirical training risk can be minimized to arbitrary small value and the estimation error that related to  $N_p$  asymptotically tends to 0 as the sample size  $N_p$  tends to infinity. Thanks to the large amount of pretrained samples  $N_p$ , we can approximate the generalization error bound for the model learned by ProGrad as

$$\begin{aligned} \mathcal{R}_d(\hat{f}_{\text{prograd}}) &\leq \\ \frac{1}{2} \gamma_{\mathcal{F}}(S, P) &+ \frac{cB \left( \sqrt{2 \log(2)L} + 1 \right) \prod_{j=1}^L M_j}{\sqrt{N_d}} \\ &+ \frac{3}{2} \sqrt{\frac{(b-a) \ln(4/\epsilon)}{2N_d}} + \frac{1}{2} \sqrt{\frac{(b-a)^2 \ln(4/\epsilon)}{2} \frac{1}{N_d}}. \end{aligned} \quad (14)$$

Similarly, we have the generalization error for  $\hat{f}_{\text{coop}}$  as

$$\begin{aligned} \mathcal{R}_d(\hat{f}_{\text{coop}}) &\leq 2 \frac{cB \left( \sqrt{2 \log(2)L} + 1 \right) \prod_{j=1}^L M_j}{\sqrt{N_d}} \\ &+ 3 \sqrt{\frac{(b-a) \ln(4/\epsilon)}{2N_d}} + \sqrt{\frac{(b-a)^2 \ln(4/\epsilon)}{2} \frac{1}{N_d}}. \end{aligned} \quad (15)$$

If the gap between the pre-trained domain  $\mathcal{D}_p$  and the downstream domain  $\mathcal{D}_d$  is very small, the  $\gamma_{\mathcal{F}}(D, P)$  will tend to 0. Under this assumption, the estimation error bound of  $\mathcal{R}_d(\hat{f}_{\text{coop}})$  is at least 2 times greater than  $\mathcal{R}_d(\hat{f}_{\text{prograd}})$ . Considering that in few-shot setting,  $N_d$  is typically very small, which makes our ProGrad model  $\hat{f}_{\text{prograd}}$  a much lower error bound than conventional fine-tuning model  $\hat{f}_{\text{coop}}$ .

## B. Additional Implementation Details

For ProGrad implementation, we first initialize the learnable context vector  $\mathbf{v}$  with the word embeddings of the zero-shot hand-crafted prompt. Concretely, if the context length  $M$  is 16 and the hand-crafted prompt is “a photo of a”, which only has 4 tokens, we initialize the former 12 context vectors with zeros and the last 4 context vectors with the word embedding of “a photo of a”. We follow the training settings of CoOp [55]: All prompt-based models are trained by SGD with an initial learning rate of 0.002 which is decayed by the cosine annealing rule. During the first epoch, we use the warm-up trick by fixing the learning rate to  $1 \times 10^{-5}$  to alleviate the gradient explosion. The training epoch is set to 50 for all shots of experiments of ImageNet dataset. For the rest 10 datasets, the training epoch is set to 50 for 1 shot, 100 for 2/4 shots and 200 for 8/16 shots. We train all prompt-based model with batch size of 32 expect for CoCoOp. As described in [56], CoCoOp consumes a significant amount of GPU memory if the batch size is set larger

than one. We set the batch size to 1, following their original setting. Our experiments are conducted on one 2080Ti GPU for all datasets except ImageNet where we train the models on one A100 GPU.

## C. Hand-crafted Prompts

Table 8: Hand-crafted Prompts.

| Dataset             | Hand-crafted prompt                    |
|---------------------|----------------------------------------|
| OxfordPets          | "a type of pet, a photo of a {}."      |
| OxfordFlowers       | "a type of flower, a photo of a {}."   |
| FGVCAircraft        | "a type of aircraft, a photo of a {}." |
| DescribableTextures | "a texture of {}."                     |
| EuroSAT             | "a centered satellite photo of {}."    |
| StanfordCars        | "a photo of a {}."                     |
| Food101             | "a type of food, a photo of {}."       |
| SUN397              | "a photo of a {}."                     |
| Caltech101          | "a photo of a {}."                     |
| UCF101              | "a photo of a person doing {}."        |
| ImageNet            | "a photo of a {}."                     |
| ImageNetSketch      | "a photo of a {}."                     |
| ImageNetV2          | "a photo of a {}."                     |
| ImageNetA           | "a photo of a {}."                     |
| ImageNetR           | "a photo of a {}."                     |

The hand-crafted prompts for 11 datasets as well as the ImageNet variants are listed in Table 8. We select the ensemble prompts from CLIP [37], examples for ImageNet are shown in Table 9.

## D. Additional Experiments

### D.1. Additional Few-shot Classification Results

In this section, we further provide the detailed few-shot classification results of other learning-based fine-tuning methods with confidence interval at 95% in Table 10 and Table 11.

**Cosine.** As described in Section 4.5 of the main paper, we plug in an additional cosine classifier on top of the visual backbone and trained on downstream dataset.

**CoOp** learns the context prompt from data rather than hand-crafted design.

**CLIP-Adapter** learns additional feature adapter to boost conventional fine-tuning results.

**Cosine + ProGrad** employs ProGrad to the training process of cosine classifier.

**CoOp +  $l_2$  prompt reg.** We further investigate whether simply using the  $l_2$  distance between learned prompt vector  $\mathbf{v}$  and the word embedding vector of hand-crafted prompt  $\mathbf{v}_{zs}$  as the regularization can improve few-shot performance, *i.e.*,  $\mathcal{L}_{total}(\mathbf{v}) = \mathcal{L}_{ce}(\mathbf{v}) + \alpha \|\mathbf{v} - \mathbf{v}_{zs}\|_2$ , where we select  $\alpha = 0.01$ .

**CoOp + GM** applies gradient matching method [53] to CoOp, *i.e.*, we not only project the  $\mathbf{G}_d$  to the perpendicular direction of  $\mathbf{G}_g$  as the updated gradient, but also project

the  $\mathbf{G}_g$  to the perpendicular direction of  $\mathbf{G}_d$  as the updated gradient to fine-tune the model alternately.

**CoOp + KD.** As described in Section 4.5 of the main paper, we apply knowledge distillation loss to CoOp, *i.e.*,  $\mathcal{L}_{total} = \mathcal{L}_{ce} + \mathcal{L}_{kl}$

**ProGrad Upper Bound** first optimizes a prompt with plain cross-entropy loss on the full dataset to create  $\mathbf{G}_g$  and then use such gradient to implement ProGrad.

For all prompt-based methods, we set the context length  $M$  to 16 except for CoOp +  $l_2$  prompt reg. The learned length for CoOp +  $l_2$  prompt reg needs to be equal to the hand-crafted prompt length to compute the  $l_2$  norm, *e.g.*, the  $M$  has to be 4 if the hand-crafted prompt is "a photo of a {}". According to the average results in Table 10, we observe that our CoOp + ProGrad still achieves the best average performance. By comparing the results of 1) Cosine and Cosine + ProGrad; and 2) CoOp and CoOp + ProGrad, we demonstrates both conventional "pre-train then fine-tune" paradigm and prompt tuning paradigm can benefit from our ProGrad. The gap between CoOp and CoOp +  $l_2$  prompt reg demonstrates that directly regularize the learned prompt to be not far away from the hand-crafted prompt has limited improvement. By digging into CoOp + KD and CoOp + GM, we find performance improvement by introducing the general knowledge. However, their performance still underperforms our CoOp + ProGrad. This is because 1) CoOp + KD learns the average knowledge from two domains which still allows the fine-tuned model to learn from the downstream knowledge that conflicts with the general knowledge; 2) CoOp + MD additional requires the fine-tuned model to discards the general knowledge that is not aligned with the downstream knowledge, as the downstream data is limited, the inaccurate estimation of  $\mathbf{G}_d$  will lead the model focus on biased general knowledge.

### D.2. Additional Results for Base-to-New Generalization

Table 12 further presents the results for base-to-new generalization on each of the 11 datasets.

### D.3. Additional Results for Domain Generalization

Table 13 further provides the averaged accuracies with standard deviations for domain generalization setting.

## References

- [1] Peter L Bartlett and Shahar Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3(Nov):463–482, 2002. 4
- [2] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101—mining discriminative components with random forests. In *ECCV*, 2014. 5

Table 9: Prompt Ensembling Examples for ImageNet.

---

"a bad photo of a {}." "a photo of many {}." "a sculpture of a {}."  
 "a photo of the hard to see {}." "a low resolution photo of the {}."  
 "a rendering of a {}." "graffiti of a {}." "a bad photo of the {}."  
 "a cropped photo of the {}." "a tattoo of a {}." "the embroidered {}."  
 "a photo of a hard to see {}." "a bright photo of a {}."  
 "a photo of a clean {}." "a photo of a dirty {}."  
 "a dark photo of the {}." "a drawing of a {}."  
 "a photo of my {}." "the plastic {}." "a photo of the cool {}."  
 "a close-up photo of a {}." "a black and white photo of the {}."  
 "a painting of the {}." "a painting of a {}."  
 "a pixelated photo of the {}." "a sculpture of the {}."  
 "a bright photo of the {}." "a cropped photo of a {}." "a plastic {}."  
 "a photo of the dirty {}." "a jpeg corrupted photo of a {}."  
 "a blurry photo of the {}." "a photo of the {}." "a good photo of the {}."  
 "a rendering of the {}." "a {} in a video game." "a photo of one {}."  
 "a doodle of a {}." "a close-up photo of the {}." "a photo of a {}."  
 "the origami {}." "the {} in a video game." "a sketch of a {}."  
 "a doodle of the {}." "a origami {}." "a low resolution photo of a {}."  
 "the toy {}." "a rendition of the {}." "a photo of the clean {}."  
 "a photo of a large {}." "a rendition of a {}." "a photo of a nice {}."  
 "a photo of a weird {}." "a blurry photo of a {}." "a cartoon {}."  
 "art of a {}." "a sketch of the {}." "a embroidered {}."  
 "a pixelated photo of a {}." "itap of the {}."  
 "a jpeg corrupted photo of the {}." "a good photo of a {}."  
 "a plushie {}." "a photo of the nice {}." "a photo of the small {}."  
 "a photo of the weird {}." "the cartoon {}." "art of the {}."  
 "a drawing of the {}." "a photo of the large {}."  
 "a black and white photo of a {}." "the plushie {}."  
 "a dark photo of a {}." "itap of a {}."  
 "graffiti of the {}." "a toy {}." "itap of my {}."  
 "a photo of a cool {}." "a photo of a small {}." "a tattoo of the {}."

---

- [3] Tony Cai, Jianqing Fan, and Tiejing Jiang. Distributions of angles in random packing on spheres. *Journal of Machine Learning Research*, 14(21):1837–1864, 2013. [8](#)
- [4] Guangyi Chen, Weiran Yao, Xiangchen Song, Xinyue Li, Yongming Rao, and Kun Zhang. Prompt learning with optimal transport for vision-language models. *arXiv preprint arXiv:2210.01253*, 2022. [3](#)
- [5] Mircea Cimpoi, Subhansu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *CVPR*, 2014. [5](#)
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. [5](#)
- [7] Mohammad Mahdi Derakhshani, Enrique Sanchez, Adrian Bulat, Victor Guilherme Turrisi da Costa, Cees GM Snoek, Georgios Tzimiropoulos, and Brais Martinez. Variational prompt tuning improves generalization of vision-language models. *arXiv preprint arXiv:2210.02390*, 2022. [3](#)
- [8] Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal gradient descent for continual learning. In *International Conference on Artificial Intelligence and Statistics*, 2020. [3](#)
- [9] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *CVPRW*, 2004. [5](#)
- [10] Peng Gao, Shijie Geng, Renrui Zhang, Teli Ma, Rongyao Fang, Yongfeng Zhang, Hongsheng Li, and Yu Qiao. Clip-adaptor: Better vision-language models with feature adapters. *arXiv preprint arXiv:2110.04544*, 2021. [3](#), [5](#), [15](#), [16](#)
- [11] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. In *ACL*, 2021. [1](#)
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. [5](#)
- [13] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019. [5](#)
- [14] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, et al. The many faces of robust-

- ness: A critical analysis of out-of-distribution generalization. In *ICCV*, 2021. 5
- [15] Dan Hendrycks, Kevin Zhao, Steven Basart, Jacob Steinhardt, and Dawn Song. Natural adversarial examples. In *CVPR*, 2021. 5
- [16] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, et al. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2(7), 2015. 4
- [17] Xinting Hu, Kaihua Tang, Chunyan Miao, Xian-Sheng Hua, and Hanwang Zhang. Distilling causal effect of data in class-incremental learning. In *CVPR*, 2021. 3
- [18] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *ICML*, 2021. 1
- [19] Woojeong Jin, Yu Cheng, Yelong Shen, Weizhu Chen, and Xiang Ren. A good prompt is worth millions of parameters: Low-resource prompt-based learning for vision-language models. In *ACL*, 2022. 1
- [20] Muhammad Uzair Khattak, Hanoona Rasheed, Muhammad Maaz, Salman Khan, and Fahad Shahbaz Khan. Maple: Multi-modal prompt learning. *arXiv preprint arXiv:2210.03117*, 2022. 3
- [21] Wonjae Kim, Bokyung Son, and Ildoo Kim. Vilt: Vision-and-language transformer without convolution or region supervision. In *ICML*, 2021. 3
- [22] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *ICCVW*, 2013. 5
- [23] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, 2021. 1, 3
- [24] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017. 8
- [25] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *arXiv preprint arXiv:2107.13586*, 2021. 1
- [26] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too. *arXiv preprint arXiv:2103.10385*, 2021. 1, 3
- [27] Yaoyao Liu, Yuting Su, An-An Liu, Bernt Schiele, and Qianru Sun. Mnemonics training: Multi-class incremental learning without forgetting. In *CVPR*, 2020. 3
- [28] Jiasen Lu, Dhruv Batra, Devi Parikh, and Stefan Lee. Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. *NeurIPS*, 2019. 3
- [29] Yuning Lu, Jianzhuang Liu, Yonggang Zhang, Yajing Liu, and Xinmei Tian. Prompt distribution learning. In *CVPR*, 2022. 3
- [30] Subhansu Maji, Esa Rahtu, Juho Kannala, Matthew Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151*, 2013. 5
- [31] Alfred Müller. Integral probability metrics and their generating classes of functions. *Advances in Applied Probability*, 29(2):429–443, 1997. 4, 10
- [32] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, 2008. 5
- [33] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *CVPR*, 2012. 5
- [34] Mary Phuong and Christoph Lampert. Towards understanding knowledge distillation. In *ICML*, 2019. 4
- [35] Chengwei Qin and Shafiq Joty. Continual few-shot relation learning via embedding space regularization and data augmentation. In *ACL*, 2022. 3
- [36] Chengwei Qin and Shafiq Joty. Lfpt5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5. In *ICLR*, 2022. 1
- [37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 1, 3, 11
- [38] Alexandre Rame, Corentin Dancette, and Matthieu Cord. Fishr: Invariant gradient variances for out-of-distribution generalization. *arXiv preprint arXiv:2109.02934*, 2021. 3
- [39] Yongming Rao, Wenliang Zhao, Guangyi Chen, Yansong Tang, Zheng Zhu, Guan Huang, Jie Zhou, and Jiwen Lu. Denseclip: Language-guided dense prediction with context-aware prompting. *arXiv preprint arXiv:2112.01518*, 2021. 3
- [40] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *CVPR*, 2017. 3
- [41] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishal Shankar. Do imagenet classifiers generalize to imagenet? In *ICML*, 2019. 5
- [42] Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and Gerald Tesauro. Learning to learn without forgetting by maximizing transfer and minimizing interference. In *ICLR*, 2018. 3
- [43] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *CVPR*, 2017. 2
- [44] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. *NeurIPS*, 2018. 3
- [45] Yuge Shi, Jeffrey Seely, Philip Torr, Siddharth N, Awni Han-nun, Nicolas Usunier, and Gabriel Synnaeve. Gradient matching for domain generalization. In *ICLR*, 2022. 3
- [46] Manli Shu, Weili Nie, De-An Huang, Zhiding Yu, Tom Goldstein, Anima Anandkumar, and Chaowei Xiao. Test-time prompt tuning for zero-shot generalization in vision-language models. *arXiv preprint arXiv:2209.07511*, 2022. 3
- [47] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 5

- [48] Hao Tan and Mohit Bansal. Lxmert: Learning cross-modality encoder representations from transformers. In *EMNLP-IJCNLP*, 2019. 3
- [49] Haohan Wang, Songwei Ge, Zachary Lipton, and Eric P Xing. Learning robust global representations by penalizing local predictive power. *NeurIPS*, 2019. 5
- [50] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *CVPR*, 2010. 5
- [51] Shuo Yang, Songhua Wu, Tongliang Liu, and Min Xu. Bridging the gap between few-shot and many-shot learning via distribution calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. 10
- [52] Yuan Yao, Ao Zhang, Zhengyan Zhang, Zhiyuan Liu, Tat-Seng Chua, and Maosong Sun. Cpt: Colorful prompt tuning for pre-trained vision-language models. *arXiv preprint arXiv:2109.11797*, 2021. 1, 3
- [53] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *NeurIPS*, 2020. 3, 4, 11
- [54] Chao Zhang, Lei Zhang, and Jieping Ye. Generalization bounds for domain adaptation. *NeurIPS*, 2012. 4
- [55] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *arXiv preprint arXiv:2109.01134*, 2021. 1, 3, 4, 5, 10
- [56] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Conditional prompt learning for vision-language models. In *CVPR*, 2022. 1, 2, 3, 5, 7, 10
- [57] Beier Zhu, Yulei Niu, Saeil Lee, Minhoe Hur, and Hanwang Zhang. Debaised fine-tuning for vision-language models by prompt regularization. *AAAI*, 2023. 3

Table 10: Accuracy (%) with standard deviation of few-shot learning on 11 datasets (**Part I**). The context length  $M$  is set 16 for prompt-based methods. \* indicates results copied from [10].

|              | Method                  | #shots per class |                  |                  |                  |                   |
|--------------|-------------------------|------------------|------------------|------------------|------------------|-------------------|
|              |                         | 1                | 2                | 4                | 8                | 16                |
| Average      | Cosine                  | 30.50 $\pm$ 1.24 | 43.74 $\pm$ 1.37 | 53.33 $\pm$ 1.57 | 61.26 $\pm$ 1.45 | 65.00 $\pm$ 2.87  |
|              | CoOp                    | 59.44 $\pm$ 1.88 | 62.31 $\pm$ 1.40 | 66.72 $\pm$ 0.93 | 70.06 $\pm$ 0.53 | 73.48 $\pm$ 0.39  |
|              | CLIP-Adapter*           | 61.45            | 64.32            | 67.51            | 70.78            | 74.35             |
|              | Cosine + ProGrad        | 32.29 $\pm$ 1.12 | 46.14 $\pm$ 1.49 | 55.18 $\pm$ 1.99 | 62.05 $\pm$ 0.93 | 66.47 $\pm$ 1.69  |
|              | CoOp + $l_2$ prompt reg | 60.84 $\pm$ 1.16 | 62.75 $\pm$ 1.18 | 66.85 $\pm$ 0.76 | 70.08 $\pm$ 0.58 | 72.92 $\pm$ 0.46  |
|              | CoOp + GM               | 61.27 $\pm$ 0.96 | 63.23 $\pm$ 0.50 | 64.59 $\pm$ 0.63 | 66.40 $\pm$ 0.49 | 67.12 $\pm$ 0.29  |
|              | CoOp + KD               | 61.52 $\pm$ 0.99 | 64.07 $\pm$ 0.52 | 66.52 $\pm$ 0.38 | 70.01 $\pm$ 0.31 | 72.01 $\pm$ 0.37  |
|              | ProGrad Upper Bound     | 74.14 $\pm$ 0.51 | 74.24 $\pm$ 0.29 | 74.52 $\pm$ 0.52 | 75.20 $\pm$ 0.35 | 76.36 $\pm$ 0.25  |
|              | ProGrad                 | 62.61 $\pm$ 0.80 | 64.90 $\pm$ 0.86 | 68.45 $\pm$ 0.52 | 71.41 $\pm$ 0.49 | 74.28 $\pm$ 0.40  |
|              |                         |                  |                  |                  |                  |                   |
| ImageNet     | Cosine                  | 15.95 $\pm$ 0.07 | 26.56 $\pm$ 0.30 | 37.08 $\pm$ 0.29 | 46.18 $\pm$ 0.19 | 53.36 $\pm$ 0.39  |
|              | CoOp                    | 57.15 $\pm$ 1.03 | 57.25 $\pm$ 0.43 | 59.51 $\pm$ 0.25 | 61.59 $\pm$ 0.17 | 63.00 $\pm$ 0.18  |
|              | CLIP-Adapter*           | 58.14            | 58.55            | 59.41            | 60.36            | 61.27             |
|              | Cosine + ProGrad        | 19.21 $\pm$ 0.28 | 31.18 $\pm$ 0.18 | 42.59 $\pm$ 0.29 | 51.73 $\pm$ 0.18 | 57.65 $\pm$ 0.33  |
|              | CoOp + $l_2$ prompt reg | 57.51 $\pm$ 0.22 | 61.27 $\pm$ 0.49 | 62.49 $\pm$ 0.12 | 62.71 $\pm$ 0.01 | 62.88 $\pm$ 0.09  |
|              | CoOp + GM               | 60.41 $\pm$ 0.17 | 60.51 $\pm$ 0.13 | 60.75 $\pm$ 0.06 | 61.01 $\pm$ 0.14 | 61.44 $\pm$ 0.03  |
|              | CoOp + KD               | 60.85 $\pm$ 0.22 | 61.08 $\pm$ 0.10 | 61.51 $\pm$ 0.07 | 61.67 $\pm$ 0.12 | 62.05 $\pm$ 0.09  |
|              | ProGrad Upper Bound     | 61.28 $\pm$ 0.19 | 61.60 $\pm$ 0.14 | 62.42 $\pm$ 0.16 | 63.5 $\pm$ 0.15  | 64.34 $\pm$ 0.11  |
|              | ProGrad                 | 57.75 $\pm$ 0.24 | 59.75 $\pm$ 0.33 | 61.46 $\pm$ 0.07 | 62.54 $\pm$ 0.03 | 63.45 $\pm$ 0.08  |
|              |                         |                  |                  |                  |                  |                   |
| Caltech101   | Cosine                  | 60.76 $\pm$ 1.71 | 73.10 $\pm$ 1.01 | 81.43 $\pm$ 0.65 | 87.02 $\pm$ 0.60 | 90.60 $\pm$ 0.05  |
|              | CoOp                    | 87.40 $\pm$ 0.98 | 87.92 $\pm$ 1.12 | 89.48 $\pm$ 0.47 | 90.25 $\pm$ 0.18 | 92.00 $\pm$ 0.02  |
|              | CLIP-Adapter*           | 88.52            | 89.19            | 91.04            | 91.71            | 93.42             |
|              | Cosine + ProGrad        | 61.95 $\pm$ 0.12 | 75.24 $\pm$ 0.88 | 82.98 $\pm$ 0.38 | 88.59 $\pm$ 0.21 | 91.31 $\pm$ 0.19  |
|              | CoOp + $l_2$ prompt reg | 87.04 $\pm$ 0.61 | 87.37 $\pm$ 0.78 | 88.82 $\pm$ 0.40 | 89.62 $\pm$ 0.29 | 91.67 $\pm$ 0.26  |
|              | CoOp + GM               | 89.14 $\pm$ 0.15 | 89.37 $\pm$ 0.26 | 89.64 $\pm$ 0.33 | 89.36 $\pm$ 0.31 | 89.42 $\pm$ 0.13  |
|              | CoOp + KD               | 89.06 $\pm$ 0.29 | 89.71 $\pm$ 0.20 | 90.13 $\pm$ 0.16 | 90.09 $\pm$ 0.30 | 91.39 $\pm$ 0.05  |
|              | ProGrad Upper Bound     | 91.08 $\pm$ 0.11 | 91.70 $\pm$ 0.30 | 91.76 $\pm$ 0.52 | 91.84 $\pm$ 0.16 | 92.86 $\pm$ 0.07  |
|              | ProGrad                 | 88.68 $\pm$ 0.34 | 87.98 $\pm$ 0.69 | 89.99 $\pm$ 0.26 | 90.83 $\pm$ 0.07 | 92.17 $\pm$ 0.17  |
|              |                         |                  |                  |                  |                  |                   |
| OxfordPets   | Cosine                  | 26.33 $\pm$ 0.75 | 41.60 $\pm$ 1.93 | 55.29 $\pm$ 1.97 | 66.60 $\pm$ 0.82 | 66.84 $\pm$ 16.24 |
|              | CoOp                    | 86.01 $\pm$ 0.47 | 82.21 $\pm$ 2.12 | 86.63 $\pm$ 1.02 | 85.15 $\pm$ 1.12 | 87.06 $\pm$ 0.88  |
|              | CLIP-Adapter*           | 81.44            | 81.57            | 82.69            | 84.13            | 85.31             |
|              | Cosine + ProGrad        | 26.08 $\pm$ 0.73 | 40.58 $\pm$ 2.01 | 55.23 $\pm$ 1.44 | 66.78 $\pm$ 1.58 | 68.96 $\pm$ 14.35 |
|              | CoOp + $l_2$ prompt reg | 87.55 $\pm$ 0.15 | 82.12 $\pm$ 2.61 | 84.93 $\pm$ 1.77 | 84.38 $\pm$ 0.75 | 86.28 $\pm$ 0.45  |
|              | CoOp + GM               | 87.05 $\pm$ 0.65 | 87.06 $\pm$ 0.67 | 88.45 $\pm$ 0.45 | 88.35 $\pm$ 0.15 | 88.38 $\pm$ 0.27  |
|              | CoOp + KD               | 87.10 $\pm$ 1.47 | 87.40 $\pm$ 0.60 | 88.56 $\pm$ 0.19 | 88.77 $\pm$ 0.24 | 89.16 $\pm$ 0.16  |
|              | ProGrad Upper Bound     | 88.56 $\pm$ 0.30 | 87.82 $\pm$ 0.78 | 87.99 $\pm$ 0.80 | 88.02 $\pm$ 0.40 | 88.88 $\pm$ 0.31  |
|              | ProGrad                 | 88.36 $\pm$ 0.73 | 86.89 $\pm$ 0.42 | 88.04 $\pm$ 0.50 | 87.91 $\pm$ 0.54 | 89.00 $\pm$ 0.32  |
|              |                         |                  |                  |                  |                  |                   |
| StanfordCars | Cosine                  | 18.96 $\pm$ 0.34 | 33.37 $\pm$ 0.38 | 47.75 $\pm$ 0.38 | 61.30 $\pm$ 0.25 | 71.94 $\pm$ 0.31  |
|              | CoOp                    | 55.68 $\pm$ 1.23 | 58.33 $\pm$ 0.60 | 63.05 $\pm$ 0.09 | 68.37 $\pm$ 0.25 | 73.34 $\pm$ 0.49  |
|              | CLIP-Adapter*           | 56.02            | 58.24            | 63.07            | 67.00            | 72.83             |
|              | Cosine + ProGrad        | 21.13 $\pm$ 0.50 | 39.44 $\pm$ 0.83 | 54.54 $\pm$ 0.57 | 66.47 $\pm$ 0.14 | 73.41 $\pm$ 0.11  |
|              | CoOp + $l_2$ prompt reg | 55.86 $\pm$ 0.66 | 57.69 $\pm$ 0.51 | 62.82 $\pm$ 0.07 | 66.63 $\pm$ 0.25 | 69.86 $\pm$ 0.44  |
|              | CoOp + GM               | 57.37 $\pm$ 0.36 | 58.46 $\pm$ 0.24 | 59.72 $\pm$ 0.66 | 62.32 $\pm$ 0.59 | 63.87 $\pm$ 0.37  |
|              | CoOp + KD               | 57.48 $\pm$ 1.47 | 59.09 $\pm$ 0.60 | 61.47 $\pm$ 0.19 | 67.73 $\pm$ 0.24 | 70.48 $\pm$ 0.16  |
|              | ProGrad Upper Bound     | 70.54 $\pm$ 0.14 | 71.57 $\pm$ 0.16 | 71.66 $\pm$ 0.50 | 72.73 $\pm$ 0.15 | 75.27 $\pm$ 0.36  |
|              | ProGrad                 | 58.38 $\pm$ 0.23 | 61.81 $\pm$ 0.45 | 65.62 $\pm$ 0.43 | 69.29 $\pm$ 0.11 | 73.46 $\pm$ 0.29  |
|              |                         |                  |                  |                  |                  |                   |
| Flowers102   | Cosine                  | 51.33 $\pm$ 2.77 | 70.06 $\pm$ 2.29 | 82.43 $\pm$ 1.65 | 91.74 $\pm$ 0.73 | 95.68 $\pm$ 0.22  |
|              | CoOp                    | 68.13 $\pm$ 1.74 | 76.68 $\pm$ 1.82 | 86.13 $\pm$ 0.75 | 91.74 $\pm$ 0.49 | 94.72 $\pm$ 0.34  |
|              | CLIP-Adapter*           | 71.97            | 78.80            | 85.31            | 90.69            | 94.30             |
|              | Cosine + ProGrad        | 52.08 $\pm$ 2.31 | 70.13 $\pm$ 1.90 | 81.09 $\pm$ 2.06 | 91.62 $\pm$ 0.41 | 93.94 $\pm$ 0.02  |
|              | CoOp + $l_2$ prompt reg | 71.12 $\pm$ 0.55 | 80.36 $\pm$ 0.54 | 86.42 $\pm$ 0.33 | 91.58 $\pm$ 0.59 | 94.25 $\pm$ 0.38  |
|              | CoOp + GM               | 67.87 $\pm$ 0.31 | 69.09 $\pm$ 0.49 | 71.69 $\pm$ 0.68 | 75.76 $\pm$ 0.79 | 78.36 $\pm$ 0.34  |
|              | CoOp + KD               | 68.11 $\pm$ 1.47 | 71.02 $\pm$ 0.60 | 76.06 $\pm$ 0.19 | 84.53 $\pm$ 0.24 | 88.05 $\pm$ 0.16  |
|              | ProGrad Upper Bound     | 95.29 $\pm$ 0.28 | 95.29 $\pm$ 0.38 | 95.70 $\pm$ 0.38 | 95.86 $\pm$ 0.37 | 96.52 $\pm$ 0.02  |
|              | ProGrad                 | 73.18 $\pm$ 0.73 | 79.77 $\pm$ 0.65 | 85.37 $\pm$ 0.96 | 91.64 $\pm$ 0.24 | 94.37 $\pm$ 0.24  |
|              |                         |                  |                  |                  |                  |                   |

Table 11: Accuracy (%) with confidence interval at 95% of few-shot learning on 11 datasets (**Part II**). The context length  $M$  is set 16 for prompt-based methods. \* indicates results copied from [10].

|               | Method                  | #shots per class |                  |                  |                   |                   |
|---------------|-------------------------|------------------|------------------|------------------|-------------------|-------------------|
|               |                         | 1                | 2                | 4                | 8                 | 16                |
| Food101       | Cosine                  | 25.32 $\pm$ 0.29 | 41.06 $\pm$ 0.29 | 54.10 $\pm$ 1.06 | 61.88 $\pm$ 0.33  | 68.50 $\pm$ 0.24  |
|               | CoOp                    | 74.28 $\pm$ 1.40 | 72.45 $\pm$ 1.29 | 73.27 $\pm$ 2.07 | 71.67 $\pm$ 0.30  | 74.68 $\pm$ 0.03  |
|               | CLIP-Adapter*           | 75.09            | 75.59            | 75.92            | 76.53             | 76.97             |
|               | Cosine + ProGrad        | 27.19 $\pm$ 0.15 | 45.28 $\pm$ 0.36 | 58.57 $\pm$ 1.01 | 71.25 $\pm$ 0.29  | 75.61 $\pm$ 0.15  |
|               | CoOp + $l_2$ prompt reg | 73.58 $\pm$ 2.20 | 68.89 $\pm$ 1.30 | 71.30 $\pm$ 0.49 | 72.42 $\pm$ 0.26  | 75.64 $\pm$ 0.33  |
|               | CoOp + GM               | 76.23 $\pm$ 1.51 | 77.97 $\pm$ 0.51 | 78.89 $\pm$ 0.10 | 78.90 $\pm$ 0.15  | 79.07 $\pm$ 0.06  |
|               | CoOp + KD               | 76.06 $\pm$ 1.47 | 77.59 $\pm$ 0.60 | 78.72 $\pm$ 0.19 | 78.38 $\pm$ 0.24  | 78.90 $\pm$ 0.16  |
|               | ProGrad Upper Bound     | 79.35 $\pm$ 0.15 | 78.19 $\pm$ 0.25 | 77.67 $\pm$ 0.51 | 78.05 $\pm$ 0.42  | 78.99 $\pm$ 0.14  |
|               | ProGrad                 | 76.04 $\pm$ 0.54 | 74.95 $\pm$ 0.57 | 75.95 $\pm$ 0.27 | 76.65 $\pm$ 0.23  | 78.41 $\pm$ 0.08  |
|               |                         |                  |                  |                  |                   |                   |
| FGVC Aircraft | Cosine                  | 12.47 $\pm$ 1.00 | 17.75 $\pm$ 1.35 | 22.00 $\pm$ 1.50 | 29.14 $\pm$ 0.54  | 36.47 $\pm$ 0.18  |
|               | CoOp                    | 9.71 $\pm$ 6.09  | 18.74 $\pm$ 0.48 | 21.78 $\pm$ 0.50 | 27.55 $\pm$ 0.06  | 31.37 $\pm$ 0.53  |
|               | CLIP-Adapter*           | 19.63            | 22.27            | 25.62            | 30.48             | 38.72             |
|               | Cosine + ProGrad        | 12.83 $\pm$ 0.48 | 17.59 $\pm$ 1.59 | 19.70 $\pm$ 1.62 | 26.34 $\pm$ 0.51  | 31.98 $\pm$ 0.68  |
|               | CoOp + $l_2$ prompt reg | 18.01 $\pm$ 0.44 | 19.78 $\pm$ 0.23 | 22.51 $\pm$ 0.94 | 27.24 $\pm$ 0.38  | 30.55 $\pm$ 0.54  |
|               | CoOp + GM               | 17.08 $\pm$ 0.37 | 19.34 $\pm$ 0.24 | 19.62 $\pm$ 0.40 | 21.07 $\pm$ 0.08  | 22.52 $\pm$ 0.19  |
|               | CoOp + KD               | 17.67 $\pm$ 0.45 | 19.29 $\pm$ 0.15 | 21.21 $\pm$ 0.60 | 25.55 $\pm$ 0.30  | 28.58 $\pm$ 0.42  |
|               | ProGrad Upper Bound     | 28.83 $\pm$ 0.09 | 28.97 $\pm$ 0.12 | 30.44 $\pm$ 0.79 | 31.92 $\pm$ 0.9   | 34.32 $\pm$ 0.16  |
|               | ProGrad                 | 18.81 $\pm$ 0.50 | 20.47 $\pm$ 0.90 | 23.32 $\pm$ 0.36 | 27.02 $\pm$ 0.67  | 31.12 $\pm$ 0.62  |
|               |                         |                  |                  |                  |                   |                   |
| SUN397        | Cosine                  | 25.32 $\pm$ 0.18 | 38.13 $\pm$ 0.37 | 49.83 $\pm$ 0.45 | 56.97 $\pm$ 0.21  | 62.84 $\pm$ 0.16  |
|               | CoOp                    | 60.30 $\pm$ 0.64 | 59.52 $\pm$ 0.60 | 63.33 $\pm$ 0.39 | 65.65 $\pm$ 0.10  | 69.14 $\pm$ 0.11  |
|               | CLIP-Adapter*           | 61.16            | 62.08            | 64.74            | 66.88             | 69.20             |
|               | Cosine + ProGrad        | 29.66 $\pm$ 0.08 | 45.81 $\pm$ 0.39 | 55.92 $\pm$ 0.35 | 63.61 $\pm$ 0.16  | 67.33 $\pm$ 0.25  |
|               | CoOp + $l_2$ prompt reg | 57.64 $\pm$ 0.33 | 59.81 $\pm$ 0.33 | 64.88 $\pm$ 0.45 | 67.66 $\pm$ 0.16  | 69.56 $\pm$ 0.11  |
|               | CoOp + GM               | 62.73 $\pm$ 0.35 | 62.85 $\pm$ 0.10 | 63.32 $\pm$ 0.21 | 63.77 $\pm$ 0.04  | 64.47 $\pm$ 0.27  |
|               | CoOp + KD               | 62.89 $\pm$ 0.40 | 64.10 $\pm$ 0.29 | 65.83 $\pm$ 0.26 | 67.02 $\pm$ 0.05  | 68.32 $\pm$ 0.19  |
|               | ProGrad Upper Bound     | 67.65 $\pm$ 0.33 | 66.89 $\pm$ 0.44 | 68.33 $\pm$ 0.36 | 68.46 $\pm$ 0.24  | 70.18 $\pm$ 0.33  |
|               | ProGrad                 | 60.54 $\pm$ 0.24 | 63.06 $\pm$ 0.11 | 66.39 $\pm$ 0.43 | 67.62 $\pm$ 0.28  | 69.84 $\pm$ 0.18  |
|               |                         |                  |                  |                  |                   |                   |
| DTD           | Cosine                  | 27.05 $\pm$ 0.83 | 38.42 $\pm$ 0.48 | 48.44 $\pm$ 2.29 | 58.47 $\pm$ 0.51  | 61.88 $\pm$ 0.38  |
|               | CoOp                    | 43.77 $\pm$ 2.12 | 46.06 $\pm$ 1.05 | 53.82 $\pm$ 0.77 | 60.06 $\pm$ 1.18  | 63.26 $\pm$ 0.22  |
|               | CLIP-Adapter*           | 45.65            | 50.54            | 56.43            | 61.59             | 66.03             |
|               | Cosine + ProGrad        | 26.95 $\pm$ 1.38 | 38.87 $\pm$ 1.02 | 48.05 $\pm$ 3.02 | 56.24 $\pm$ 2.81  | 63.40 $\pm$ 0.58  |
|               | CoOp + $l_2$ prompt reg | 43.74 $\pm$ 1.45 | 45.98 $\pm$ 2.76 | 53.25 $\pm$ 1.55 | 59.08 $\pm$ 0.58  | 62.31 $\pm$ 1.05  |
|               | CoOp + GM               | 43.81 $\pm$ 2.15 | 47.64 $\pm$ 0.63 | 49.17 $\pm$ 1.52 | 53.17 $\pm$ 0.63  | 54.06 $\pm$ 0.45  |
|               | CoOp + KD               | 43.01 $\pm$ 2.18 | 49.31 $\pm$ 1.10 | 53.03 $\pm$ 1.49 | 60.26 $\pm$ 0.34  | 63.14 $\pm$ 0.39  |
|               | ProGrad Upper Bound     | 66.11 $\pm$ 0.77 | 67.04 $\pm$ 0.72 | 67.24 $\pm$ 0.34 | 68.46 $\pm$ 0.41  | 69.27 $\pm$ 0.64  |
|               | ProGrad                 | 46.14 $\pm$ 1.74 | 49.78 $\pm$ 1.37 | 54.43 $\pm$ 0.86 | 60.69 $\pm$ 0.10  | 63.97 $\pm$ 0.61  |
|               |                         |                  |                  |                  |                   |                   |
| EuroSAT       | Cosine                  | 37.55 $\pm$ 5.27 | 52.93 $\pm$ 5.66 | 49.81 $\pm$ 6.23 | 46.08 $\pm$ 11.13 | 33.30 $\pm$ 13.04 |
|               | CoOp                    | 49.40 $\pm$ 3.86 | 62.23 $\pm$ 4.94 | 69.49 $\pm$ 3.23 | 76.56 $\pm$ 1.73  | 84.05 $\pm$ 1.05  |
|               | CLIP-Adapter*           | 54.53            | 63.73            | 68.33            | 75.81             | 82.81             |
|               | Cosine + ProGrad        | 41.55 $\pm$ 6.19 | 51.35 $\pm$ 5.76 | 47.64 $\pm$ 9.68 | 30.03 $\pm$ 2.99  | 33.30 $\pm$ 1.67  |
|               | CoOp + $l_2$ prompt reg | 54.28 $\pm$ 5.38 | 62.60 $\pm$ 2.77 | 70.43 $\pm$ 1.81 | 77.32 $\pm$ 2.20  | 83.30 $\pm$ 1.11  |
|               | CoOp + GM               | 48.02 $\pm$ 4.04 | 57.12 $\pm$ 2.03 | 62.88 $\pm$ 2.28 | 68.74 $\pm$ 2.18  | 67.72 $\pm$ 1.00  |
|               | CoOp + KD               | 49.51 $\pm$ 1.12 | 58.89 $\pm$ 1.06 | 66.79 $\pm$ 0.76 | 74.37 $\pm$ 0.91  | 77.87 $\pm$ 1.74  |
|               | ProGrad Upper Bound     | 90.97 $\pm$ 0.36 | 89.81 $\pm$ 1.89 | 89.57 $\pm$ 0.70 | 90.19 $\pm$ 0.47  | 90.92 $\pm$ 0.35  |
|               | ProGrad                 | 56.32 $\pm$ 3.04 | 63.10 $\pm$ 3.77 | 72.53 $\pm$ 1.29 | 78.04 $\pm$ 2.45  | 83.74 $\pm$ 0.70  |
|               |                         |                  |                  |                  |                   |                   |
| UCF101        | Cosine                  | 34.41 $\pm$ 0.40 | 48.21 $\pm$ 1.00 | 58.47 $\pm$ 0.81 | 68.46 $\pm$ 0.66  | 73.64 $\pm$ 0.32  |
|               | CoOp                    | 62.03 $\pm$ 1.13 | 63.98 $\pm$ 0.91 | 67.45 $\pm$ 0.74 | 72.11 $\pm$ 0.29  | 75.67 $\pm$ 0.49  |
|               | CLIP-Adapter*           | 63.80            | 66.98            | 70.07            | 73.45             | 76.99             |
|               | Cosine + ProGrad        | 36.61 $\pm$ 0.14 | 52.11 $\pm$ 1.43 | 60.66 $\pm$ 1.50 | 69.85 $\pm$ 0.94  | 74.27 $\pm$ 0.30  |
|               | CoOp + $l_2$ prompt reg | 62.88 $\pm$ 0.74 | 64.43 $\pm$ 0.71 | 67.46 $\pm$ 0.40 | 72.28 $\pm$ 0.88  | 75.77 $\pm$ 0.29  |
|               | CoOp + GM               | 64.27 $\pm$ 0.48 | 66.14 $\pm$ 0.25 | 66.37 $\pm$ 0.27 | 67.91 $\pm$ 0.29  | 68.96 $\pm$ 0.04  |
|               | CoOp + KD               | 64.99 $\pm$ 0.35 | 67.29 $\pm$ 0.46 | 68.44 $\pm$ 0.13 | 71.77 $\pm$ 0.41  | 74.15 $\pm$ 0.55  |
|               | ProGrad Upper Bound     | 76.99 $\pm$ 0.42 | 77.24 $\pm$ 0.47 | 76.95 $\pm$ 0.66 | 78.16 $\pm$ 0.17  | 78.44 $\pm$ 0.26  |
|               | ProGrad                 | 64.55 $\pm$ 0.50 | 66.35 $\pm$ 0.18 | 69.86 $\pm$ 0.30 | 73.33 $\pm$ 0.65  | 77.28 $\pm$ 0.96  |
|               |                         |                  |                  |                  |                   |                   |

Table 12: Accuracy (%) for the base-to-new generalization evaluation. The context length  $M$  is 4 for prompt-based methods which are learned from the base classes with 4 shots. H: Harmonic mean.

| (a) Average over 11 datasets. |              |              |              | (b) ImageNet.   |              |              | (c) Caltech101. |  |              | (d) OxfordPets.    |              |              |
|-------------------------------|--------------|--------------|--------------|-----------------|--------------|--------------|-----------------|--|--------------|--------------------|--------------|--------------|
|                               | Base         | New          | H            |                 | Base         | New          | H               |  | Base         | New                | H            |              |
| CLIP                          | 61.72        | 65.91        | 63.64        |                 | 64.46        | 59.99        | 62.14           |  | 90.90        | 90.72              | 90.81        | 85.86        |
| CoOp                          | 71.96        | 61.26        | 65.58        |                 | 65.49        | 57.70        | 61.35           |  | 94.38        | 87.48              | 90.80        | 90.31        |
| CoCoOp                        | 72.23        | 60.77        | 65.35        |                 | 66.21        | 58.01        | 61.84           |  | 94.43        | 87.81              | 91.00        | 89.07        |
| ProGrad                       | <b>73.29</b> | <b>65.96</b> | <b>69.06</b> |                 | <b>66.96</b> | <b>60.04</b> | <b>63.23</b>    |  | <b>94.47</b> | <b>90.84</b>       | <b>92.46</b> | <b>91.78</b> |
| (e) StanfordCars.             |              |              |              | (f) Flowers102. |              |              | (g) Food101.    |  |              | (h) FGVC Aircraft. |              |              |
|                               | Base         | New          | H            |                 | Base         | New          | H               |  | Base         | New                | H            |              |
| CLIP                          | 55.55        | <b>66.35</b> | 60.47        |                 | 64.10        | <b>70.92</b> | 67.34           |  | 81.48        | 82.15              | 81.81        | 17.89        |
| CoOp                          | 61.77        | 62.51        | 62.14        |                 | <b>89.33</b> | 62.77        | 73.73           |  | 80.40        | 81.09              | 80.74        | 22.53        |
| CoCoOp                        | 61.68        | 59.98        | 60.82        |                 | 88.07        | 66.26        | 75.62           |  | 79.77        | 77.68              | 78.71        | 22.73        |
| ProGrad                       | <b>63.01</b> | 64.32        | <b>63.66</b> |                 | 88.19        | 69.38        | <b>77.66</b>    |  | <b>83.10</b> | <b>83.57</b>       | <b>83.33</b> | <b>22.77</b> |
| (i) SUN397.                   |              |              |              | (j) DTD.        |              |              | (k) EuroSAT.    |  |              | (l) UCF101.        |              |              |
|                               | Base         | New          | H            |                 | Base         | New          | H               |  | Base         | New                | H            |              |
| CLIP                          | 66.45        | <b>70.17</b> | 68.26        |                 | 49.31        | <b>54.35</b> | 51.71           |  | 39.26        | 43.62              | 41.33        | 63.70        |
| CoOp                          | 71.48        | 65.57        | 68.40        |                 | 67.71        | 43.92        | 53.28           |  | 73.53        | 40.19              | 51.97        | 74.59        |
| CoCoOp                        | 71.88        | 67.10        | 69.41        |                 | 63.54        | 40.78        | 49.68           |  | 83.63        | 40.95              | 54.98        | 73.51        |
| ProGrad                       | <b>73.71</b> | 69.78        | <b>71.69</b> |                 | <b>66.90</b> | 53.06        | <b>59.18</b>    |  | <b>79.67</b> | <b>49.99</b>       | <b>61.43</b> | <b>75.66</b> |

Table 13: Domain generalization results with standard deviation.

| (a) ResNet50  |                         |                         |                         |                         |                         |
|---------------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
|               | Source                  |                         | Target                  |                         |                         |
|               | ImageNet                | ImageNet-V2             | ImageNet-Sketch         | ImageNet-A              | ImageNet-R              |
| CoOp          | 61.34 $\pm$ 0.11        | 53.81 $\pm$ 0.10        | 32.83 $\pm$ 0.30        | 22.08 $\pm$ 0.59        | 54.62 $\pm$ 0.74        |
| CoCoOp        | 61.04 $\pm$ 0.18        | 53.71 $\pm$ 0.26        | 32.30 $\pm$ 0.34        | 22.07 $\pm$ 0.34        | 53.60 $\pm$ 0.27        |
| ProGrad       | <b>62.17</b> $\pm$ 0.06 | <b>54.70</b> $\pm$ 0.18 | <b>34.40</b> $\pm$ 0.18 | <b>23.05</b> $\pm$ 0.13 | <b>56.77</b> $\pm$ 0.33 |
| (b) ResNet101 |                         |                         |                         |                         |                         |
|               | Source                  |                         | Target                  |                         |                         |
|               | ImageNet                | ImageNet-V2             | ImageNet-Sketch         | ImageNet-A              | ImageNet-R              |
| CoOp          | 63.99 $\pm$ 0.13        | 56.99 $\pm$ 0.21        | 39.40 $\pm$ 0.29        | 29.50 $\pm$ 0.56        | 64.04 $\pm$ 0.27        |
| CoCoOp        | 63.59 $\pm$ 0.22        | 56.98 $\pm$ 0.25        | 39.16 $\pm$ 0.36        | 29.09 $\pm$ 0.18        | 64.14 $\pm$ 0.01        |
| ProGrad       | <b>64.98</b> $\pm$ 0.15 | <b>57.86</b> $\pm$ 0.04 | <b>40.53</b> $\pm$ 0.17 | <b>30.13</b> $\pm$ 0.09 | <b>65.61</b> $\pm$ 0.08 |
| (c) ViT-B/32  |                         |                         |                         |                         |                         |
|               | Source                  |                         | Target                  |                         |                         |
|               | ImageNet                | ImageNet-V2             | ImageNet-Sketch         | ImageNet-A              | ImageNet-R              |
| CoOp          | 64.74 $\pm$ 0.14        | 56.59 $\pm$ 0.27        | 40.03 $\pm$ 0.64        | 31.10 $\pm$ 0.06        | 64.54 $\pm$ 0.52        |
| CoCoOp        | 64.63 $\pm$ 0.15        | 56.59 $\pm$ 0.04        | 40.74 $\pm$ 0.24        | 30.27 $\pm$ 0.05        | 64.12 $\pm$ 0.10        |
| ProGrad       | <b>65.36</b> $\pm$ 0.23 | <b>57.42</b> $\pm$ 0.33 | <b>41.73</b> $\pm$ 0.25 | <b>31.89</b> $\pm$ 0.26 | <b>66.53</b> $\pm$ 0.08 |
| (d) ViT-B/16  |                         |                         |                         |                         |                         |
|               | Source                  |                         | Target                  |                         |                         |
|               | ImageNet                | ImageNet-V2             | ImageNet-Sketch         | ImageNet-A              | ImageNet-R              |
| CoOp          | 69.86 $\pm$ 0.22        | 62.83 $\pm$ 0.37        | 46.90 $\pm$ 0.59        | 48.98 $\pm$ 0.33        | 74.55 $\pm$ 0.46        |
| CoCoOp        | 70.13 $\pm$ 0.23        | 63.05 $\pm$ 0.06        | 46.48 $\pm$ 0.17        | 49.36 $\pm$ 0.26        | 73.80 $\pm$ 0.08        |
| ProGrad       | <b>70.45</b> $\pm$ 0.16 | <b>63.35</b> $\pm$ 0.08 | <b>48.17</b> $\pm$ 0.10 | <b>49.45</b> $\pm$ 0.08 | <b>75.21</b> $\pm$ 0.32 |